Computer and Network Security (2026-2027)

Part 2: Symmetric encryption

Jeroen Famaey
jeroen.famaey@uantwerpen.be

University of Antwerp
Faculty of Science

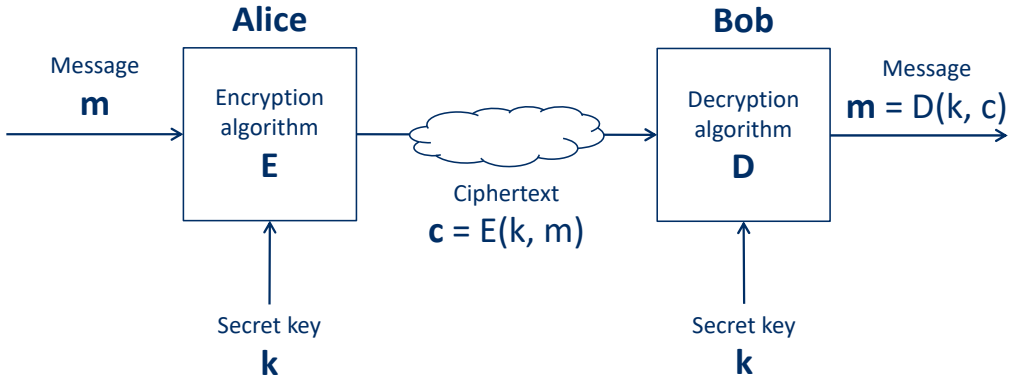
Part 2 overview

1. Symmetric encryption basics
2. Stream ciphers
   - One Time Pad
   - Practical Stream Ciphers
   - Modern Stream Cipher Examples
3. Block ciphers
   - General Block Cipher Structure
   - Advanced Encryption Standard (AES)
   - AES details
4. Block operation modes

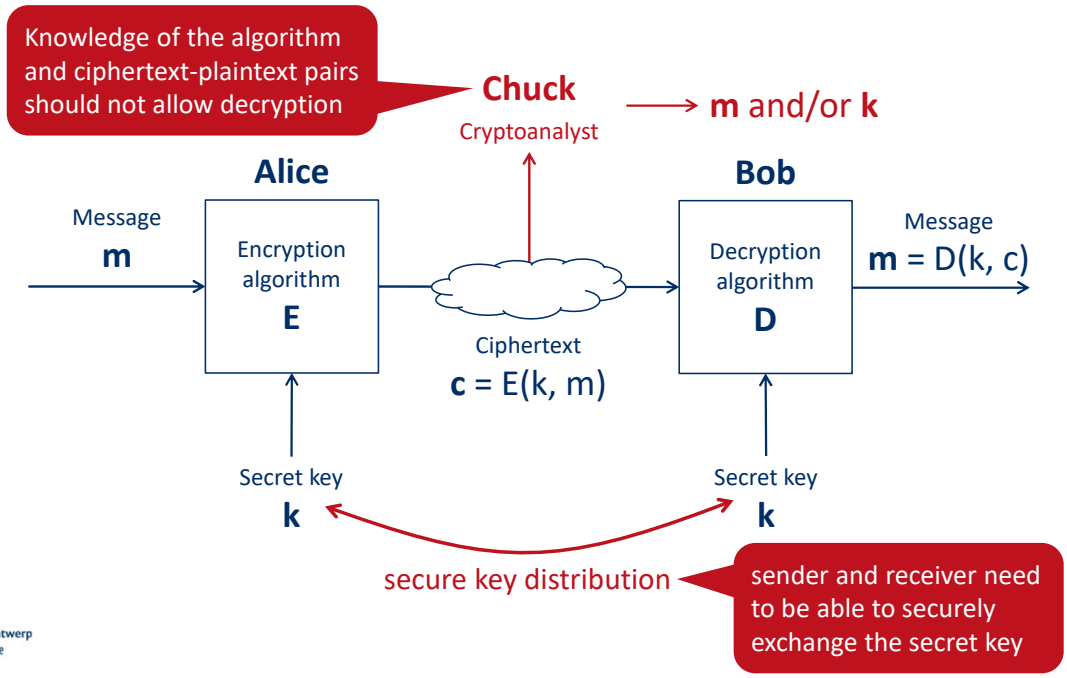
Symmetric Encryption Basics

Ensuring data confidentiality using encryption

- Also called conventional or single-key encryption
- The only type of encryption used before the 1970s
- Still the most widely used type of encryption (DES, AES, etc.)



Two requirements for encryption algorithms

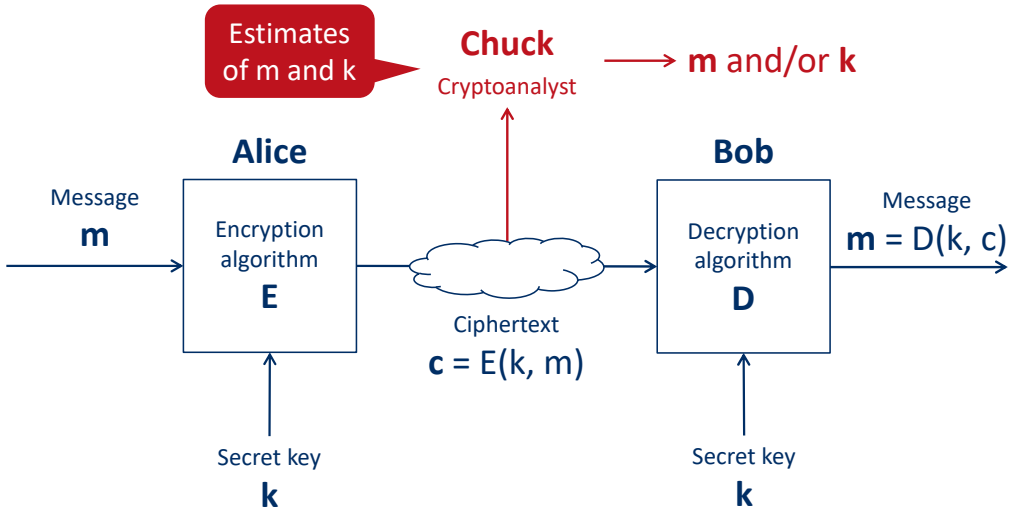


Mathematical notation

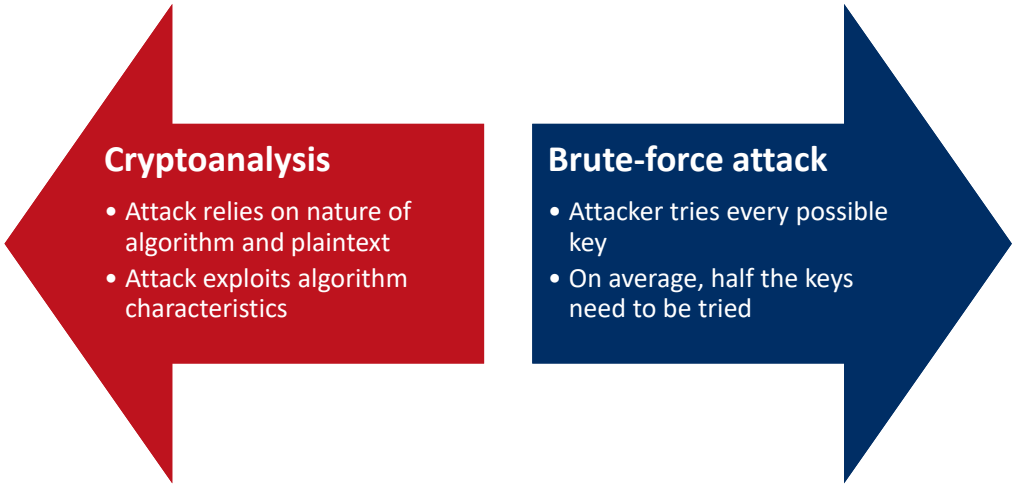
- Symmetric encryption is defined over a triple $(\boldsymbol{K}, \boldsymbol{M}, \boldsymbol{C})$:
  - Key space $\boldsymbol{K}$ contains the set of all keys
  - Message space $\boldsymbol{M}$ contains the set of all message
  - Ciphertext space $\boldsymbol{C}$ contains the set of all ciphertexts
- It consists of a set of "efficient" algorithms $(\boldsymbol{E}, \boldsymbol{D})$
  - $\boldsymbol{E}: \boldsymbol{K} \times \boldsymbol{M} \rightarrow \boldsymbol{C}$
  - $\boldsymbol{D}: \boldsymbol{K} \times \boldsymbol{C} \rightarrow \boldsymbol{M}$
- Subject to correctness property:
  - $\forall m \in \boldsymbol{M}, k \in \boldsymbol{K}: \boldsymbol{D}\big(k, \boldsymbol{E}(k, m)\big) = m$
- "Efficient" may refer to theoretical (e.g., polynomial time complexity) or practical (e.g., a maximum execution time) efficiency.

Cracking the code

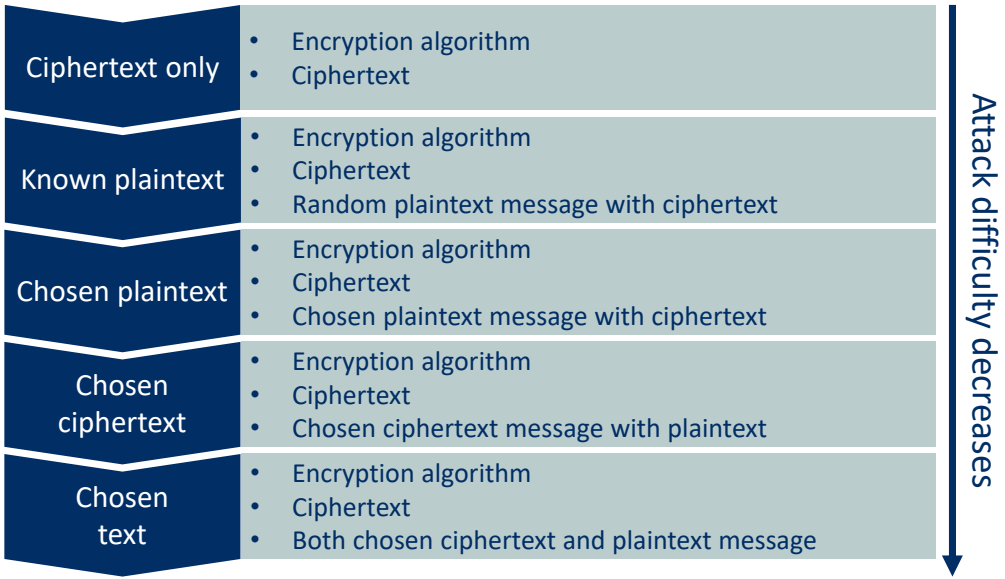
The goal is to recover the plaintext and/or key based on knowledge of c, a set of (m, c) pairs, and/or E



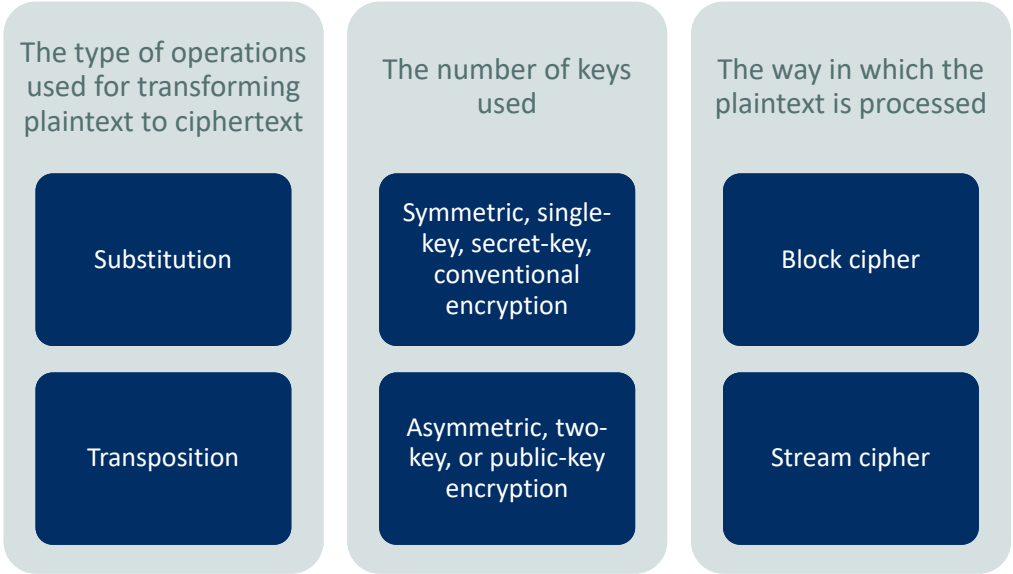
Two categories of cryptography attacks



Attacker knowledge



Characterization of cryptographic systems



Some history: the Caesar cipher



Julius Caesar
(100BC - 44BC)

- Replace each letter in the alphabet with the letter 3 places further (round robin)

- **Mapping**
  - plain: a b c d e f g h i j k l m n o p q r s t u v w x y z
  - cipher: D E F G H I J K L M N O P Q R S T U V W X Y Z A B C

- **Mathematical representation**
  - Assign each sorted letter a number in the range $[0, 25]$
  - $c = E(3, p) = (p + 3) \bmod 26$ $\quad p = D(3, c) = (c - 3) \bmod 26$

- **General Caesar algorithm**
  - $c = E(k, p) = (p + k) \bmod N$ with N letters and $k \in [1, N-1]$

What is the key-space size of a general Caesar cipher with 26 possible letters?

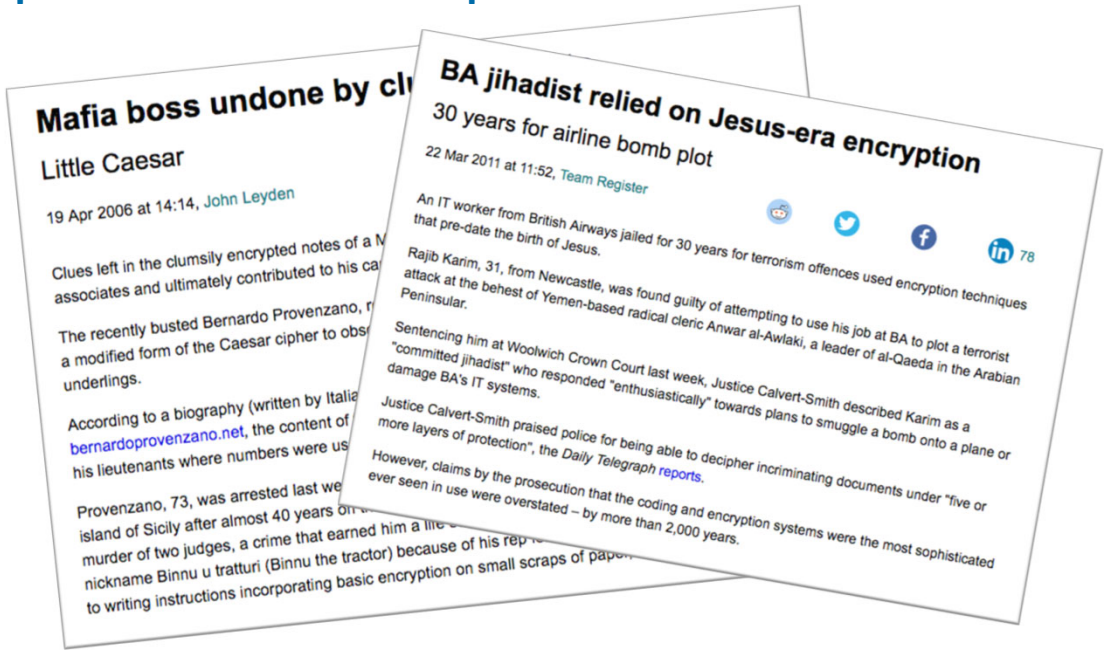
25	0%
625 (or 25^2)	0%
33 x 10^6 (or 2^25)	0%
15 x 10^24 (or 25!)	0%

What makes it easy to brute force Caesar ciphers?

KEY	PHHW PH DIWHU WKH WRJD SDUWB
1	oggv og chvgt vjg vqic rctva
2	nffu nf bgufs uif uphb qbsuz
3	meet me after the toga party
4	ldds ld zesdq sgd snfz ozqsx
5	kccr kc ydrcp rfc rmey nyprw
6	jbbq jb xcqbo qeb qldx mxoqv
7	iaap ia wbpan pda pkcw lwnpu
8	hzzo hz vaozm ocz ojbv kvmot
9	gyyn gy uznyl nby niau julns
10	fxxm fx tymxk max mhzt itkmr
11	ewwl ew sxlwj lzw lgys hsjlq
12	dvvk dv rwkvi kyv kfxr grikp
13	cuuj cu qvjuh jxu jewq fqhjo
14	btti bt puitg iwt idvp epgin
15	assh as othsf hvs hcuo dofhm
16	zrrg zr nsgre gur gbtn cnegl
17	yqqf yq mrfqd ftq fasm bmdfk
18	xppe xp lqepc esp ezrl alcej
19	wood wo kpdob dro dyqk zkbdi
20	vnnc vn jocna cqn cxpj yjach
21	ummb um inbmz bpm bwoi xizbg
22	tlla tl hmaly aol avnh whyaf
23	skkz sk glzkx znk zumg vgxze
24	rjjy rj fkyjw ymj ytlf ufwyd
25	qiix qi ejxiv xli xske tevxc

1. The algorithm is known
2. There are only 25 keys (if you exclude the trivial 0 key)
3. The plaintext language is easily recognized

Yet people still use the Caesar cipher…



Mafia boss undone by cl

Little Caesar

19 Apr 2006 at 14:14, John Leyden

Clues left in the clumsily encrypted notes of a M
associates and ultimately contributed to his ca

The recently busted Bernardo Provenzano, r
a modified form of the Caesar cipher to obs
underlings.

According to a biography (written by Italia
bernardoprovenzano.net, the content of
his lieutenants where numbers were us

Provenzano, 73, was arrested last we
island of Sicily after almost 40 years on
murder of two judges, a crime that earned him a life
nickname Binnu u tratturi (Binnu the tractor) because of his rep
to writing instructions incorporating basic encryption on small scraps of pape

BA jihadist relied on Jesus-era encryption

30 years for airline bomb plot

22 Mar 2011 at 11:52, Team Register

78

An IT worker from British Airways jailed for 30 years for terrorism offences used encryption techniques that pre-date the birth of Jesus.

Rajib Karim, 31, from Newcastle, was found guilty of attempting to use his job at BA to plot a terrorist attack at the behest of Yemen-based radical cleric Anwar al-Awlaki, a leader of al-Qaeda in the Arabian Peninsular.

Sentencing him at Woolwich Crown Court last week, Justice Calvert-Smith described Karim as a "committed jihadist" who responded "enthusiastically" towards plans to smuggle a bomb onto a plane or damage BA's IT systems.

Justice Calvert-Smith praised police for being able to decipher incriminating documents under "five or more layers of protection", the *Daily Telegraph* reports.

However, claims by the prosecution that the coding and encryption systems were the most sophisticated ever seen in use were overstated – by more than 2,000 years.

General substitution (or monoalphabetic) cipher

- Increases the key-space size compared to the Caesar cipher
- Instead of shifting the letters k spots, each plain letter is assigned a unique random letter from the alphabet

- Example substitution cipher
  - Plain: `a b c d e f g h i j k l m n o p q r s t u v w x y z`
  - Cipher: `D H U J E I X K M Q N T F Y O B P G A R L Z S V C W`

- Encryption example
  - Message: `hello my name is jeroen`
  - Ciphertext: `KETTO FC YDFE MA QEGOEY`

When poll is active respond at **PollEv.com/jfamaey** Send **jfamaey** to **+32 460 20 00 56**

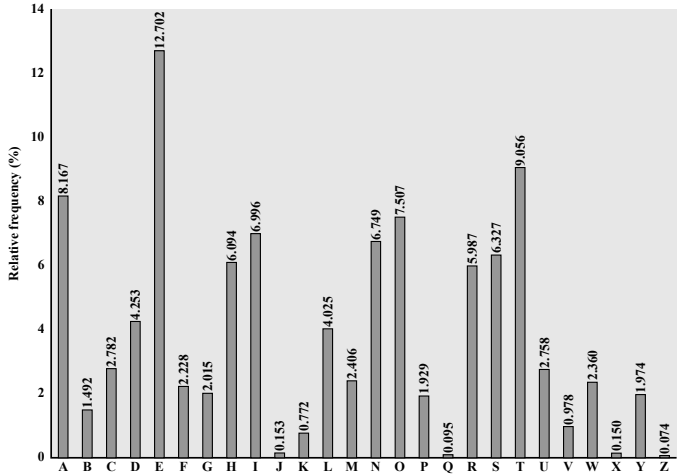
What is the key-space size of a general substitution cipher with 26 possible letters?

0

- 26
- 676 (or 26^2)
- 6.7 x 10^7 (or 2^26)
- 4 x 10^26 (or 26!)

Breaking the substitution cipher

The relative frequency of the letters can be used to map cipher letters onto one or a few potential plain letters



An example

- Ciphertext

```
UzqSovUoHxmoPvgPozPevSgzWSzoPfPeSxUDBmeT
SxaIzvUePHzHmDzSHzoWSfPaPPDTSvPqUzWymxUz
UHSxePyePoPDzSzUfPomBzWPfUPzHmDJUDTmoHmq
```

- Frequencies of letters in ciphertext

P 13.33	H 5.83	F 3.33	B 1.67	C 0.00
Z 11.67	D 5.00	W 3.33	G 1.67	K 0.00
S 8.33	E 5.00	Q 2.50	Y 1.67	L 0.00
U 8.33	V 4.17	T 2.50	I 0.83	N 0.00
O 7.50	X 4.17	A 1.67	J 0.83	R 0.00
M 6.67				

- One-on-one mapping with the English language letter frequencies
  - P = e Z = t S = a …
- Requires some trial and error, as it becomes harder to guess for less frequent letters

When poll is active respond at **PollEv.com/jfamaey** Send **jfamaey** to **+32 460 20 00 56**

What type of attack is the frequency attack?

0

- Ciphertext only
- Known plaintext
- Chosen plaintext
- Chosen ciphertext
- Chosen text

Vignère: a polyalphabetic cipher (Rome, 16th century)

- Use all 26 mono-alphabetic general Caesar cipher substitution rules
- A key represents the order in which the 26 substitution rules are used to encode subsequent plaintext letters

Plaintext: $P = p_0, p_1, \ldots, p_{L-1}$

Key: $K = k_0, k_1, \ldots, k_{M-1}$

Ciphertext: $C = c_0, c_1, \ldots, c_{L-1}$

Key is generally (much) shorter than Plaintext

Encryption: $c_i = (p_i + k_{i \bmod M}) \bmod N$

Decryption: $p_i = (c_i - k_{i \bmod M}) \bmod N$

L = plaintext size, M = key size, N = alphabet size

An example of the Vignère cipher

Exercise: Derive the plaintext, given the cipher and key below

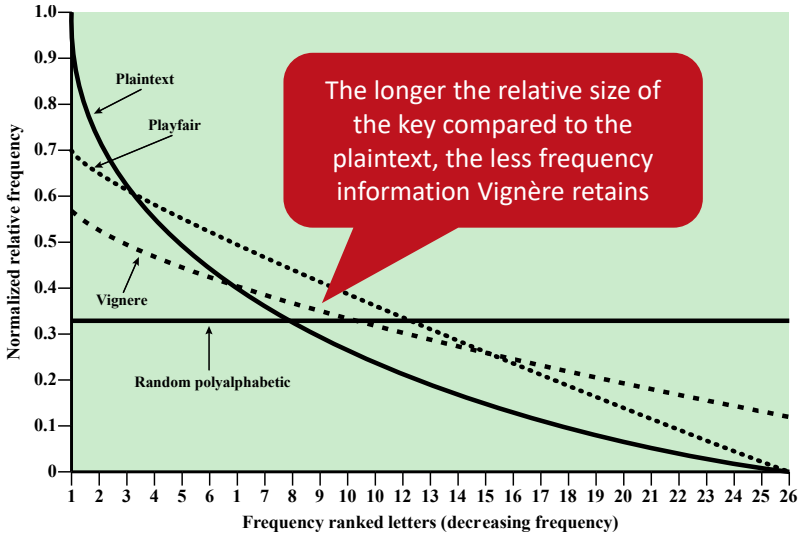
Key: `pass`

Ciphertext: `ihwscsowg`

a	b	c	d	e	f	g	h	i	j	k	l	m
0	1	2	3	4	5	6	7	8	9	10	11	12
n	**o**	**p**	**q**	**r**	**s**	**t**	**u**	**v**	**w**	**x**	**y**	**z**
13	14	15	16	17	18	19	20	21	22	23	24	25

Vignère cipher still retains frequency information

Which makes it vulnerable to some extent to frequency attacks



Cracking the Vigenère cipher

- **Determining key length**
  - When the same sequence occurs at a distance that is an integer multiple of the key length, it will generate identical ciphertext

- **Break up the cipher into m mono-alphabetic ciphers**
  - The letters encoded with the same letter from the key together form a monoalphabetic cipher that can easily be cracked with a frequency attack

When poll is active respond at **PollEv.com/jfamaey** Send **jfamaey** to **+32 460 20 00 56**

Given a ciphertext "arvsduklio" encrypted with Vigenère and a 2 letter key, which are the associated monoalphabetic ciphers?

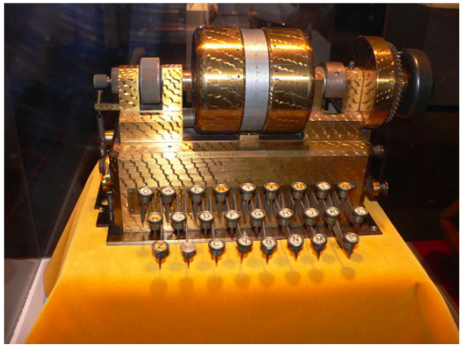
0

- "arvsd" and "uklio"
- "ar", "vs", "du", "kl" and "io"
- "avdki" and "rsulo"
- "au", "rk", "vl", "si" and "do"

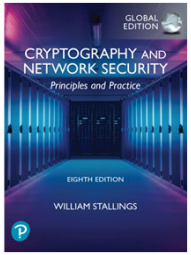
Rotor machines (1870 – 1970)

- Mechanical rotating disks substitute each typed letter based on polyalphabetic substitution cipher
- The most famous example is the German Enigma machine, whose messages were deciphered by the Allies during World War II
- Also vulnerable to frequency attacks





Further reading material



Chapter 3: Classical Encryption Techniques



Week 1: What is cryptography?

- History of cryptography

Stream ciphers

1 One Time Pad

2 Practical Stream Ciphers

3 Modern Stream Cipher Examples

Classical ciphers rely on simple substitution

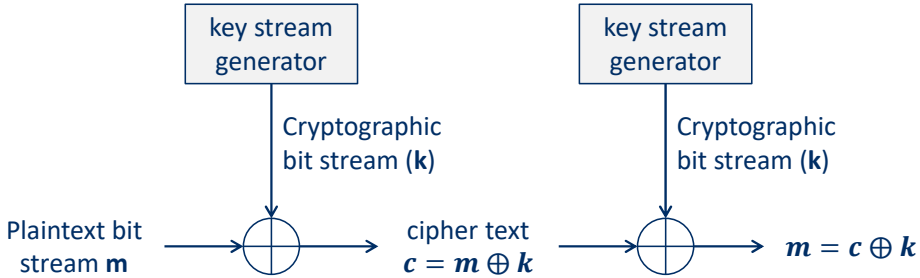
- Caesar cipher
- General substitution cipher
- Vigenere cipher
- Rotor machine
- …

→ **vulnerable to brute force or frequency attacks**

- The first secure cipher was developed by Vernam in 1917
  - Called the One Time Pad
  - $\boldsymbol{M} = \boldsymbol{C} = \{0,1\}^n$ and $\boldsymbol{K} = \{0,1\}^n$
  - Key is a random bit string with the same length as the message
  - Key is only used one time (no key reuse)

One Time Pad (OTP)

Relies on bitwise XOR operation of message and key bit streams



One Time Pad example and proof of correctness

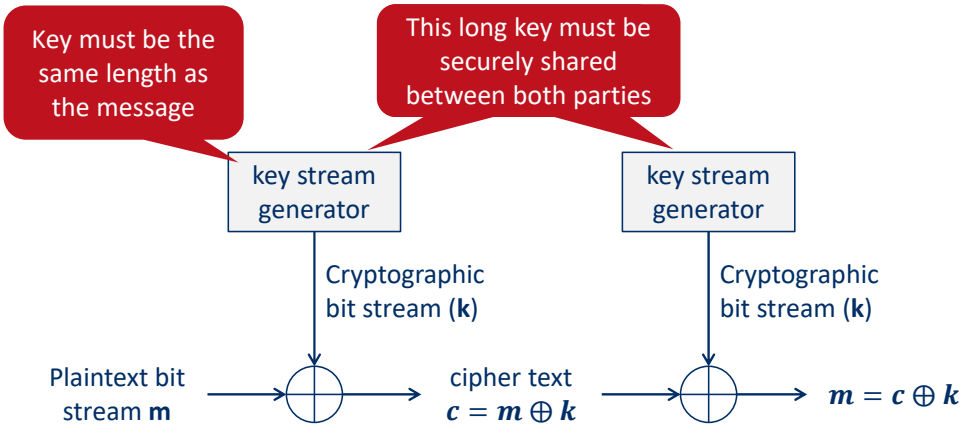
Message:		0 1 1 0 1 1 1
Key:	$\oplus$	1 0 1 1 0 0 1
Ciphertext:		1 1 0 1 1 1 0

Proof of correctness:

$$\forall m \in \boldsymbol{M}, k \in \boldsymbol{K}: \boldsymbol{D}\big(k, \boldsymbol{E}(k,m)\big) = m$$

$$D\big(k, E(k,m)\big) = D(k, k \oplus m) = k \oplus (k \oplus m) = (k \oplus k) \oplus m = m$$

Practical problems with OTP



Security definitions for encryption algorithms

Unconditional or perfect security

- The ciphertext generated by the scheme does not contain enough information to determine uniquely the corresponding plaintext, no matter how much ciphertext is available.
- $\forall m_0, m_1 \in \boldsymbol{M}, \forall c \in \boldsymbol{C}: \Pr[E(k, m_0) = c] = \Pr[E(k, m_1) = c]$
- One Time Pad is the only known unconditionally secure algorithm

Computational or semantic security

- The cost of breaking the cipher exceeds the information value, or
- The time required to break the cipher exceeds information lifetime
- $\forall m_0, m_1 \in \boldsymbol{M}, \forall c \in \boldsymbol{C}: |\Pr[E(k, m_0) = c] - \Pr[E(k, m_1) = c]| \leq \epsilon$

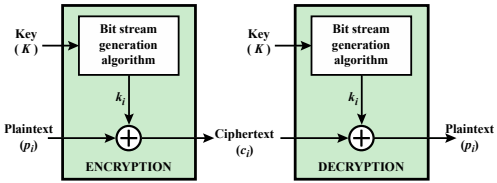
1 One Time Pad

2 Practical Stream Ciphers

3 Modern Stream Cipher Examples

Stream vs. block ciphers

Stream ciphers

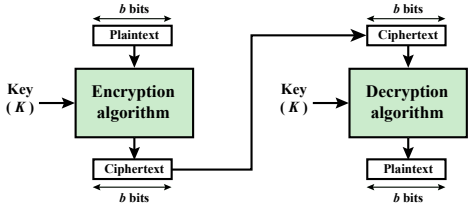


- Encrypts data one element (bit, byte, letter) at a time
- Practically, the key must be generated algorithmically

Examples

Vigenère, One Time Pad, RC4, Salsa20

Block ciphers



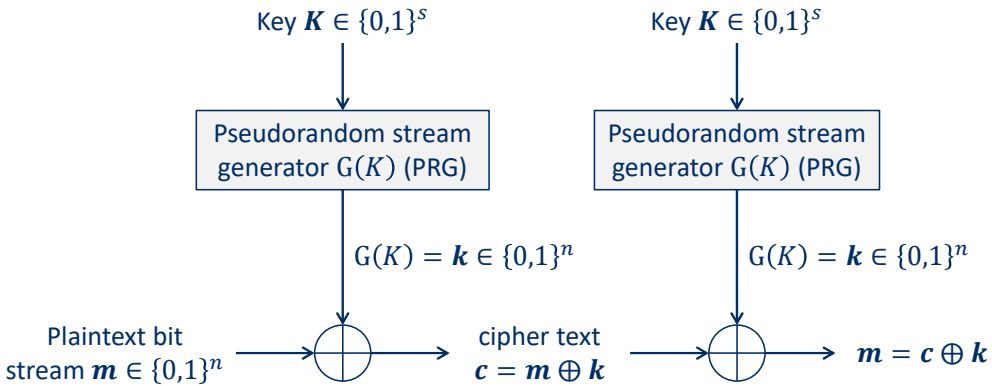
- Equal-size blocks of plaintext are encrypted as a whole
- Used by the vast majority of cryptographic applications

Examples

DES, AES

Stream ciphers: a practical version of OTP

- Idea: replace “random” key by “pseudorandom” key
- Use Pseudo-Random Generator (PRG) that turns a seed into a pseudorandom bitstream: $G: \{0,1\}^s \rightarrow \{0,1\}^n$, with $n \gg s$



Join by Web **PollEv.com/jfamaey** Join by Text Send **jfamaey** to **+32 460 20 00 56**

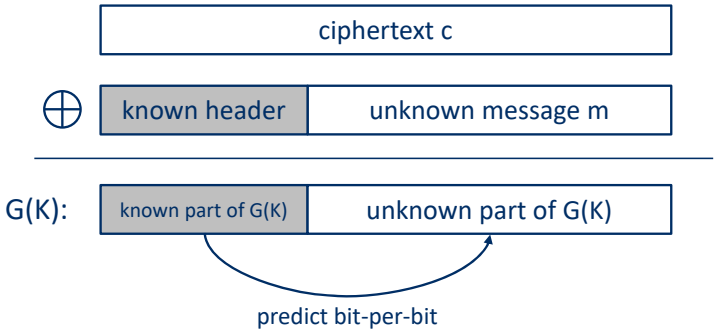
Can a stream cipher be unconditionally secure?

0

- Yes, if the PRG is secure (A)
- Yes, independent of the PRG (B)
- No (C)

PRG must be unpredictable

- We say that G: $K \rightarrow \{0,1\}^n$ is predictable if:
  - There exists an "efficient" algorithm that, given the first i bits of G(K), is able to predict bit i + 1 with a "non-negligible" probability ε
  - How to exploit predictability?



- PRG is unpredictable if it is not predictable

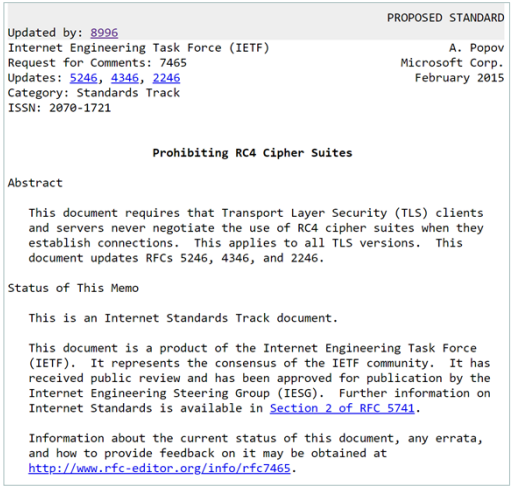
1 One Time Pad

2 Practical Stream Ciphers

3 Modern Stream Cipher Examples

RC4 stream cipher

- Designed by Ron Rivest in 1987 (one of the inventors of RSA)
- Variable key-size stream cipher with byte-oriented operations
- Was used in Wi-Fi WEP, but found to be insecure due to incorrect key generation method
- Currently still used in WPA2 and SSH
- Recent vulnerabilities (2015) have made RC4 prohibited for TLS



PROPOSED STANDARD
Updated by: 8996
Internet Engineering Task Force (IETF) — A. Popov
Request for Comments: 7465 — Microsoft Corp.
Updates: 5246, 4346, 2246 — February 2015
Category: Standards Track
ISSN: 2070-1721

Prohibiting RC4 Cipher Suites

Abstract

This document requires that Transport Layer Security (TLS) clients and servers never negotiate the use of RC4 cipher suites when they establish connections. This applies to all TLS versions. This document updates RFCs 5246, 4346, and 2246.

Status of This Memo

This is an Internet Standards Track document.

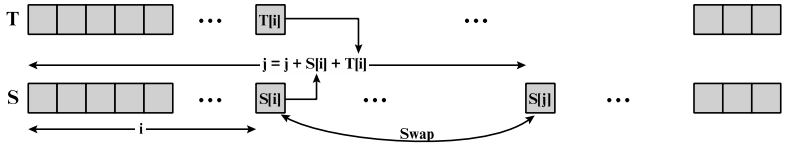
This document is a product of the Internet Engineering Task Force (IETF). It represents the consensus of the IETF community. It has received public review and has been approved for publication by the Internet Engineering Steering Group (IESG). Further information on Internet Standards is available in Section 2 of RFC 5741.

Information about the current status of this document, any errata, and how to provide feedback on it may be obtained at http://www.rfc-editor.org/info/rfc7465.

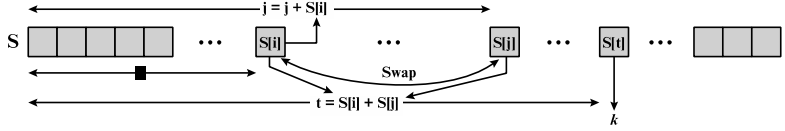
Source: *https://www.rfc-editor.org/rfc/rfc7465.html*

How does the RC4 PRG work?

- The RC4 PRG maintains an internal state S of 256 bytes
- Initialization
  - Each byte S[i] of S is initialized to the value i
  - The key K is repeated until it has 256 bytes (called T)
- T is used to create an initial permutation of S



- Stream generation
  - Each byte in S is swapped, and S[t] is selected as next output byte k




```
/* Initialization */
for i = 0 to 255 do
    S[i] = i;
    T[i] = K[i mod keylen];
```

```
/* Initial Permutation of S */
j = 0;
for i = 0 to 255 do
    j = (j + S[i] + T[i]) mod 256;
    Swap (S[i], S[j]);
```

```
/* Stream Generation */
i, j = 0;
while (true)
    i = (i + 1) mod 256;
    j = (j + S[i]) mod 256;
    Swap (S[i], S[j]);
    t = (S[i] + S[j]) mod 256;
    k = S[t];
```

Why is RC4 not secure?

- RC4 does not use a nonce, which results in vulnerabilities due to key reuse
- Like other steam ciphers, RC4 is malleable (see further)
- The first several bytes of the keystream are strongly correlated with the key
- Keystream is biased towards various sequences
  - e.g., the second output byte has a 1/128 (instead of 1/256) chance to be zero
- NOMORE (Numerous Occurrence MOnitoring & Recovery Exploit) in TLS (HTTPS) and WPA-TKIP implementation of RC4 discovered in 2014
  - First practically feasible attack against RC4 (it takes about 75 hours to execute)
  - Relies on statistical biases in the keystream (as highlighted above)

Salsa20 & ChaCha20: A more secure stream cipher

- Designed by Daniel J. Bernstein in 2005
- No attacks exist against the full 20 round variant
  - The variant performing only 8 rounds can be broken in 2250 operations [Shi et al., 2012]
- A modified version called ChaCha was published in 2008
  - Uses a new round function with better diffusion
  - Increases performance on some hardware architectures
  - Published in IETF RFC 7539
- PRG is based on add-rotate-XOR (ARX) operations, combining
  - 32-bit addition
  - fixed binary rotation (circular shift)
  - bitwise addition (XOR)
- Inputs
  - 256-bit key (128-bit key version also exists)
  - 64-bit nonce (unique (pseudo-)random number)
  - 64-bit counter

Advantages of add-rotate-XOR (ARX) ciphers

- Only use 3 operations in their round functions
  - Modular addition: $a + b \bmod 2^n$
  - Fixed binary rotation: a <<< b
  - Exclusive OR: $a \oplus b$
- These operations are computationally efficient to execute both in hardware and software

- They run in constant time
  - Avoids timing attacks (cf., lecture on asymmetric encryption for details on this)

Salsa20 PRG overview

- Transform a key K and nonce R into key stream of plaintext length n
- $\{0,1\}^{256} \times \{0,1\}^{64} \rightarrow \{0,1\}^{n}$

Key K | Nonce R | Position P

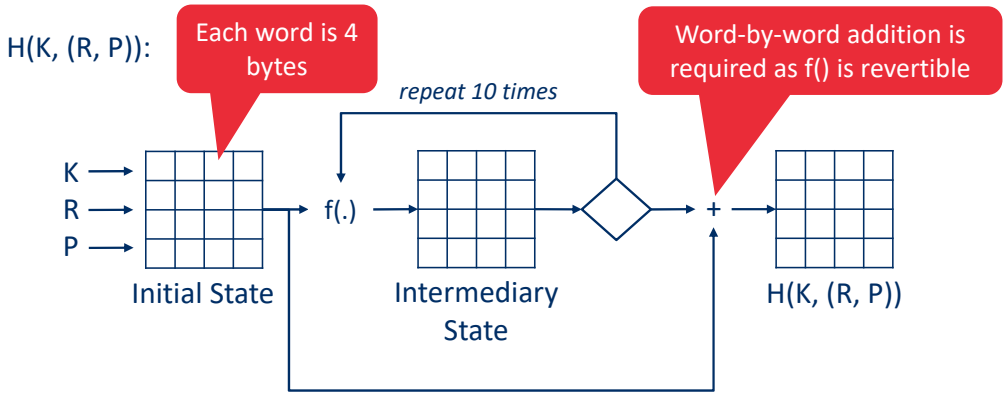
- Expansion function H(K, (R, P)):

Repeated until desired number of bytes is reached

Salsa20(K, R) = H(K, (R, 0)) || H(K, (R, 1)) || …

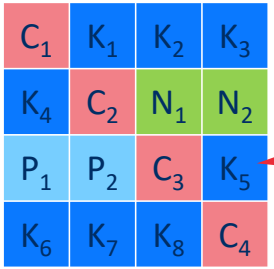
Each expansion function call increases position counter value

Expansion function H() generates a 64-byte key block



Salsa20 PRG uses internal state of 16 words

Initial state (16 words of 4 bytes):



If K is only 128 bits, then $(K_5, K_6, K_7, K_8) = (K_1, K_2, K_3, K_4)$

128-bit constant C ("expand 32-byte k" in ASCII)

256-bit key K

64-bit nonce N (pseudo-random number)

64-bit position P (counter value)

P is increased by 1 every time H() is called

Exercise: Calculating the initial 64-byte state

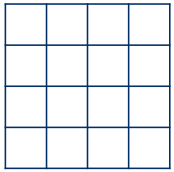
Exercise: Calculate the initial 4x4 64-byte state, given the following 128-bit key K, 64-bit nonce N, and position P.

Given:

- Constant C = `65 78 70 61 6e 64 20 31 36 2d 62 79 74 65 20 6b`
- Key K = `4f f6 9c 8e 1e 61 0a a5 c7 28 75 cd 8c 7c 69 c8`
- Nonce N = `34 1f 8a 29 05 ca 7a f8`
- Position P = `11 00 00 00 00 00 00 00` (decimal value: 17)

C_1	K_1	K_2	K_3
K_4	C_2	N_1	N_2
P_1	P_2	C_3	K_5
K_6	K_7	K_8	C_4

Determine the initial 4x4 state matrix:



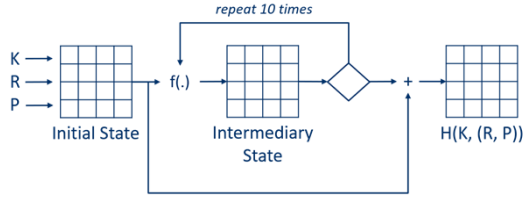
128-bit constant C ("expand 32-byte k" in ASCII)

256-bit key K

64-bit nonce N (pseudo-random number)

64-bit position P (counter value)

Round function f( )



- Each round applies the Quarterround function QR( ) twice
- QR( ) takes a 4-word input and generates a 4-word output
- First, QR( ) is applied column-per-column on the input state S_i
- Second, QR( ) is applied row-per-row

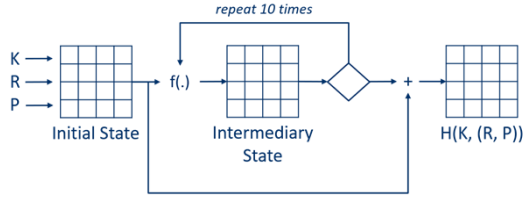
w_0	w_1	w_2	w_3
w_4	w_5	w_6	w_7
w_8	w_9	w_{10}	w_{11}
w_{12}	w_{13}	w_{14}	w_{15}

State S_i

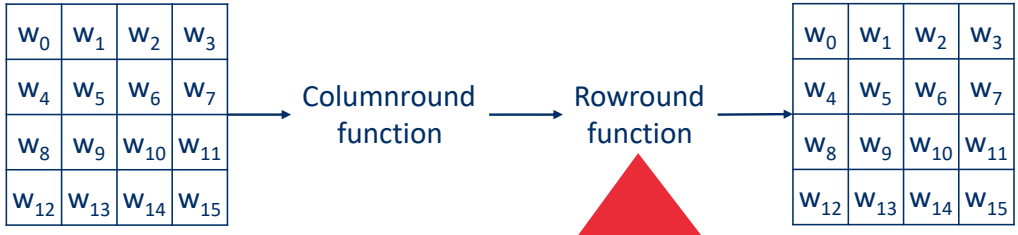
Columnround function

$$(w_0, w_4, w_8, w_{12}) = QR(w_0, w_4, w_8, w_{12})$$
$$(w_5, w_9, w_{13}, w_1) = QR(w_5, w_9, w_{13}, w_1)$$
$$(w_{10}, w_{14}, w_2, w_6) = QR(w_{10}, w_{14}, w_2, w_6)$$
$$(w_{15}, w_3, w_7, w_{11}) = QR(w_{15}, w_3, w_7, w_{11})$$

Round function f( )



- Each round applies the Quarterround function QR( ) twice
- QR( ) takes a 4-word input and generates a 4-word output
- First, QR( ) is applied column-per-column on the input state Si
- Second, QR( ) is applied row-per-row



$$(w_0, w_1, w_2, w_3) = QR(w_0, w_1, w_2, w_3)$$
$$(w_5, w_6, w_7, w_4) = QR(w_5, w_6, w_7, w_4)$$
$$(w_{10}, w_{11}, w_8, w_9) = QR(w_{10}, w_{11}, w_8, w_9)$$
$$(w_{15}, w_{12}, w_{13}, w_{14}) = QR(w_{15}, w_{12}, w_{13}, w_{14})$$

Quarter round function QR( ) using ARX

$$(y_0, y_1, y_2, y_3) = QR(x_0, x_1, x_2, x_3)$$

Where:

Word-by-word addition modulo 2^{32}

$$y_1 = x_1 \oplus ((x_0 + x_3)\text{<<<}7)$$
$$y_2 = x_2 \oplus ((y_1 + x_0)\text{<<<}9)$$
$$y_3 = x_3 \oplus ((y_2 + y_1)\text{<<<}13)$$
$$y_0 = x_0 \oplus ((y_3 + y_2)\text{<<<}18)$$

Bitwise XOR

Binary left rotation (leftmost bits are moved to the right

More information: https://cr.yp.to/snuffle/salsafamily-20071225.pdf

Exercise: Solving the quarter round function QR



Exercise: Given the four 32-bit words x_0, x_1, x_2, and x_3, calculate QR(x_0, x_1, x_2, x_3).

Given:

$$x_0 = 0xE4971786$$
$$x_1 = 0x0096E9C1$$
$$x_2 = 0x932f7534$$
$$x_3 = 0x27F0D9BB$$

Determine:

$$(y_0, y_1, y_2, y_3) = QR(x_0, x_1, x_2, x_3)$$

$$(y_0, y_1, y_2, y_3) = QR(x_0, x_1, x_2, x_3)$$

Where:

$$y_1 = x_1 \oplus ((x_0 + x_3)<<<7)$$
$$y_2 = x_2 \oplus ((y_1 + x_0)<<<9)$$
$$y_3 = x_3 \oplus ((y_2 + y_1)<<<13)$$
$$y_0 = x_0 \oplus ((y_3 + y_2)<<<18)$$

Changes in ChaCha20 variant

- Initial state consists of the same 128-bit constant and 256-bit key, but a smaller counter (32 bits) and larger nonce (96 bits), ordered differently:

C_1	C_2	C_3	C_4
K_1	K_2	K_3	K_4
K_5	K_6	K_7	K_8
P_1	N_1	N_2	N_3

- The quarter-round function $QR(x_0, x_1, x_2, x_3)$ is replaced with:

$$x_0 \mathrel{+}= x_1; x_3 = x_0 \oplus x_3; x_3 = x_3 \lll 16;$$
$$x_2 \mathrel{+}= x_3; x_1 = x_1 \oplus x_2; x_2 = x_2 \lll 12;$$
$$x_0 \mathrel{+}= x_1; x_3 = x_0 \oplus x_3; x_3 = x_3 \lll 8;$$
$$x_2 \mathrel{+}= x_3; x_1 = x_1 \oplus x_2; x_2 = x_2 \lll 7;$$

- Finally, QR is applied to diagonals in even rounds instead of rows

Stream cipher pitfall 1: You should never reuse keys

- Never use a stream cipher key more than once (!!)
- Two ciphertexts encrypted with the same key k

$$\begin{cases} c_1 \leftarrow m_1 \oplus \text{PRG}(k) \\ c_2 \leftarrow m_2 \oplus \text{PRG}(k) \end{cases}$$

- Eavesdropper can use ciphertexts to get information about plaintext:

$$c_1 \oplus c_2 = m_1 \oplus m_2$$

- By exploiting characteristics of plaintexts, attacker can recover them (e.g., using a dictionary attack):

$$m_1 \oplus m_2 \rightarrow m_1, m_2$$

- By adding a unique nonce to the key, the key itself can be reused

Stream cipher pitfall 2: Stream ciphers are malleable

- **Definition of malleability:**
  It is possible to transform a ciphertext into another ciphertext which decrypts to a related plaintext

- For a steam cipher, it is possible to calculate ciphertext $E(k, m \oplus t)$, given only the ciphertext $E(k, m)$, and without knowing the key stream $S(k)$, as:

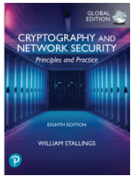
$$E(k, m) \oplus t = m \oplus S(k) \oplus t = E(k, m \oplus t)$$

- This allows performing a **bit-flipping attack**, where the attacker selects t to flip specific bits in the original message m to change its meaning
  - For example, an electronic fund transfer containing the ASCII string “1000€” can be changed into “9000€”, by calculating:

$$E(k, \text{"1000€"}) \oplus (\text{"1000€"} \oplus \text{"9000€"})$$
$$= S(k) \oplus \text{"1000€"} \oplus (\text{"1000€"} \oplus \text{"9000€"}) = S(k) \oplus \text{"9000€"}$$
$$= \mathrm{E}(k, \text{"9000€"})$$

  - Can be prevented by including a **message authentication code** to detect tampering

Further reading

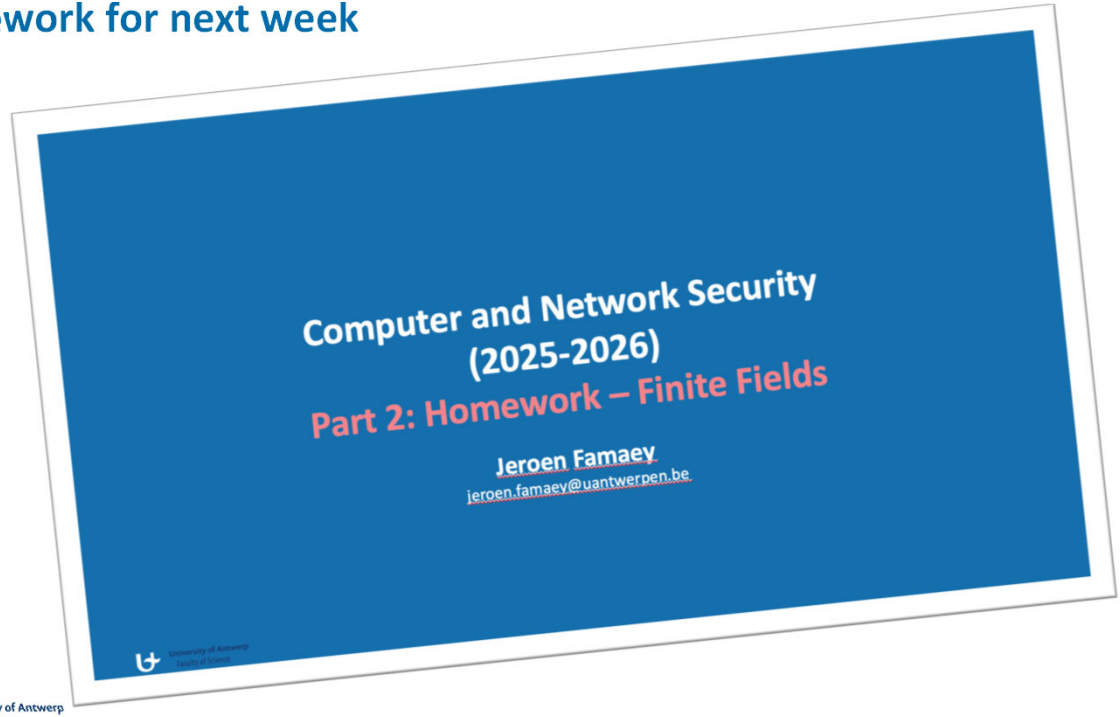


Chapter 8 (Section 8.4—8.5)



Stream ciphers (videos 1, 2, 3 and 4)

Homework for next week



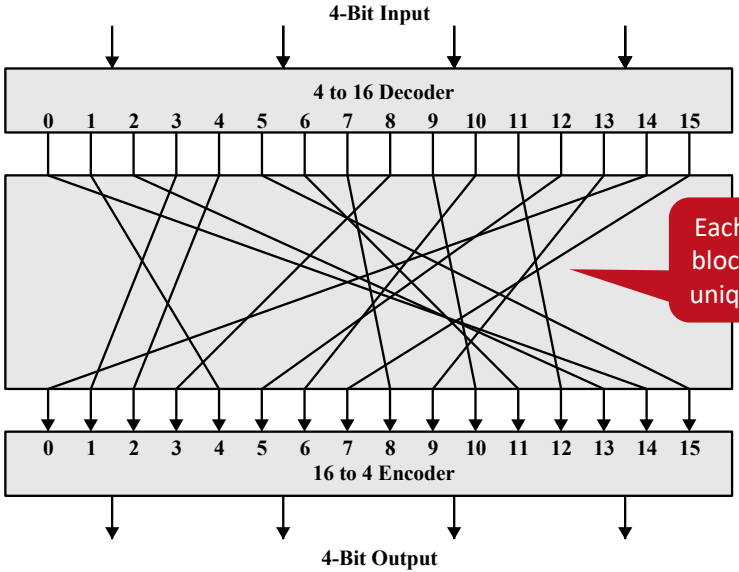
Block ciphers

1 General block cipher structure

2 Advanced Encryption Standard (AES)

3 AES details

General substitution: the ideal block cipher



The ideal block cipher has some problems

Small block size (e.g., n = 4)

It is equivalent to traditional substitution ciphers, and is vulnerable to frequency and other statistical attacks

Large block size

It is impractical, as the mapping of plain- onto ciphertext blocks needs to be captured by the key, which results in long keys

Example mapping for n = 4:

Plaintext	Ciphertext
0000	1110
0001	0100
0010	1101
0011	0001
0100	0010
0101	1111
0110	1011
0111	1000
1000	0011
1001	1010
1010	0110
1011	1100
1100	0101
1101	1001
1110	0000
1111	0111

Ciphertext	Plaintext
0000	1110
0001	0011
0010	0100
0011	1000
0100	0001
0101	1100
0110	1010
0111	1111
1000	0111
1001	1101
1010	1001
1011	0110
1100	1011
1101	0010
1110	0000
1111	0101

When poll is active respond at **PollEv.com/jfamaey** Send **jfamaey** to **+32 460 20 00 56**

What is the shortest possible key length (in bits) for the ideal block cipher with a block size n = 4?

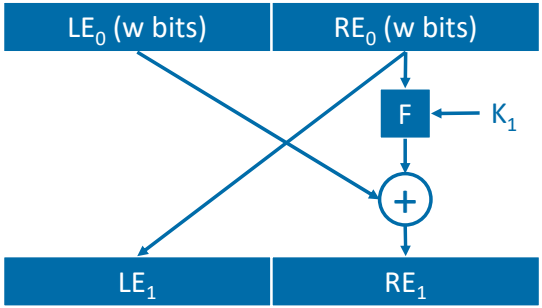
0

- 4
- 16
- 32
- 64
- 128

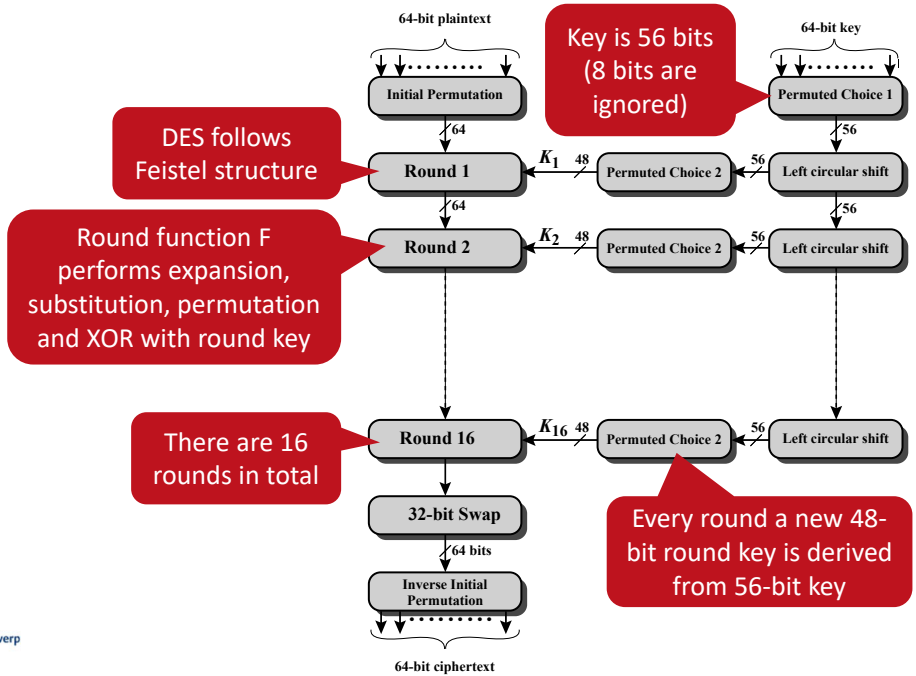
Data Encryption Standard (DES)

National Institute of Standards and Technology
U.S. Department of Commerce

- Issued in 1977 by the National Institute of Standards and Technology (NIST)
- Most used encryption scheme until introduction of AES in 2001
- DES uses 64-bit blocks and 56-bit keys, deemed unsafe
- Triple DES (3DES), which increases security through repeating the DES algorithm 3 times, was introduced in 1999
- DES is based on Feistel cipher structure:



High level overview of DES



Why use a 56 bit key?

~~(FOUO)~~ In 1973 NBS solicited private industry for a data encryption standard (DES). The first offerings were disappointing, so NSA began working on its own algorithm. Then Howard Rosenblum, deputy director for research and engineering, discovered that Walter Tuchman of IBM was working on a modification to Lucifer for general use. NSA gave Tuchman a clearance and brought him in to work jointly with the Agency on his Lucifer modification.

~~(S-CCO)~~ The decision to get involved with NBS was hardly unanimous. From the SIGINT standpoint, a competent industry standard could spread into undesirable areas, like Third World government communications, narcotics traffickers, and international terrorism targets.

This argued the opposite case – that, as Frank Rowlett had contended since World War II, in the long run it was more important to secure one's own communications than to exploit those of the enemy.[121]

~~(FOUO)~~ Once that decision had been made, the debate turned to the issue of minimizing the damage. Narrowing the encryption problem to a single, influential algorithm might drive out competitors, and that would reduce the field that NSA had to be concerned about.

NSA worked closely with IBM to strengthen the algorithm against all except brute force attacks and to strengthen substitution tables, called S-boxes. Conversely, NSA tried to convince IBM to reduce the length of the key from 64 to 48 bits. Ultimately, they compromised on a 56-bit key.[122]

Source: https://www.nsa.gov/news-features/declassified-documents/gulf-of-tonkin/articles/assets/files/release-2/rel2_american_cryptology.pdf *(page 232)*

When poll is active respond at **PollEv.com/jfamaey** Send **jfamaey** to **+32 460 20 00 56**

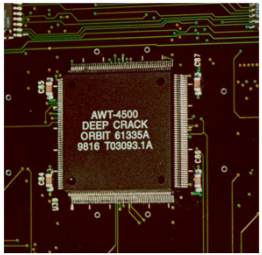
How long would it take a 2012 supercomputer to brute force a DES encrypted message on average?

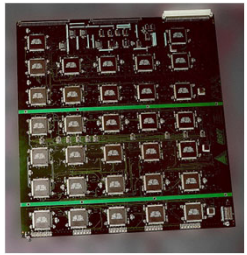
0

- 1 minute
- 1 hour
- 1 day
- 1 year
- 10 years

History of brute-force attacks against DES

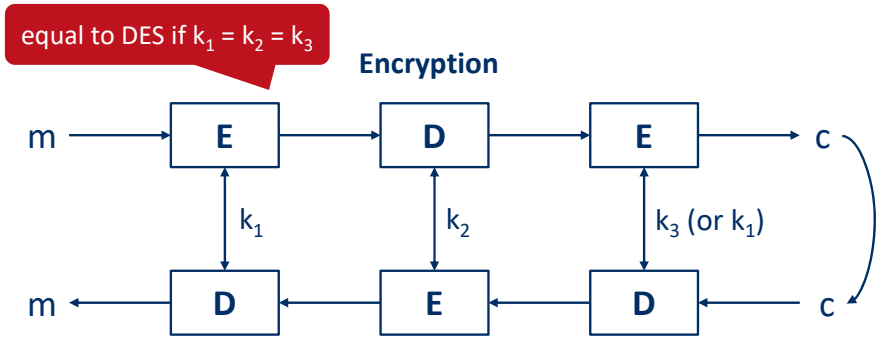
- Successful brute-force attacks have been conducted as early as 1997
- 1977: Diffie and Hellman propose a dedicated DES cracking machine which would cost $20 million and crack DES in 1 day
- 1993: Wieter proposes a key-search machine that costs $1 million and finds the key in 7 hours
- 1997: DESCHALL Project wins $10.000 prize as the first to decrypt DES using idle cycles of thousands of PCs across the Internet
- 1998: Electronic Frontier Foundation (EFF) builds first actual DES cracker for $250.000, which finds a key in about 2 days





Triple DES (3DES): A secure alternative to DES

- Employs three subsequent stages with 2 or 3 keys
- The decrypt stage allows 3DES to decrypt regular DES ciphertext
- 2-key variant reuses first key in third stage (112 bit key)
- 3-key variant uses unique key in each stage (168 bit key)



Summary up to now...

Cipher	Type	Characteristics
Caesar	Substitution	Extremely vulnerable to **brute force attacks**
Monoalphabetic	Substitution	Vulnerable to statistical **frequency attacks**
Vigenère	Substitution	Limited vulnerability to **frequency attacks**
One-time pad	Substitution	**Unconditionally secure**, but **practically infeasible**
RC4	Substitution	Not secure due to **bias in PRG**
Salsa20	Substitution	**Computationally secure** with key size of 256 bits
DES	Hybrid	Vulnerable to **brute force** due to 56-bit key size
Triple DES	Hybrid	**Computationally secure** with key size of 168 bits

1 General block cipher structure

2 Advanced Encryption Standard (AES)

3 AES details

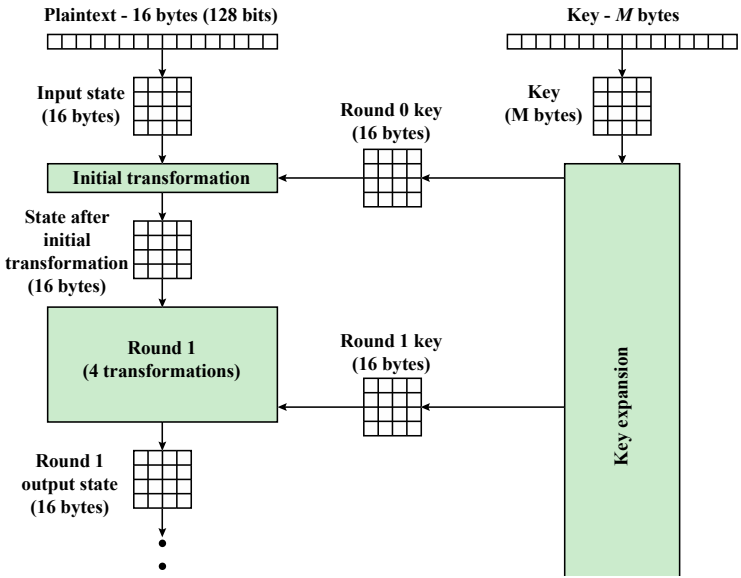
Advanced Encryption Standard history

- NIST launched an open call in 1997, and received 15 submissions
- The 5 best ones were selected in terms of:
  - Cryptographic security
  - RAM and CPU performance on PCs, embedded systems and in hardware
- In 2000 NIST announced Rijndael as the winner, which was invented by Joan Daemen and Vincent Rijmen at KU Leuven
- AES is based on a version of Rijndael with block size 128 bits and key lengths 128, 192 and 256 bits



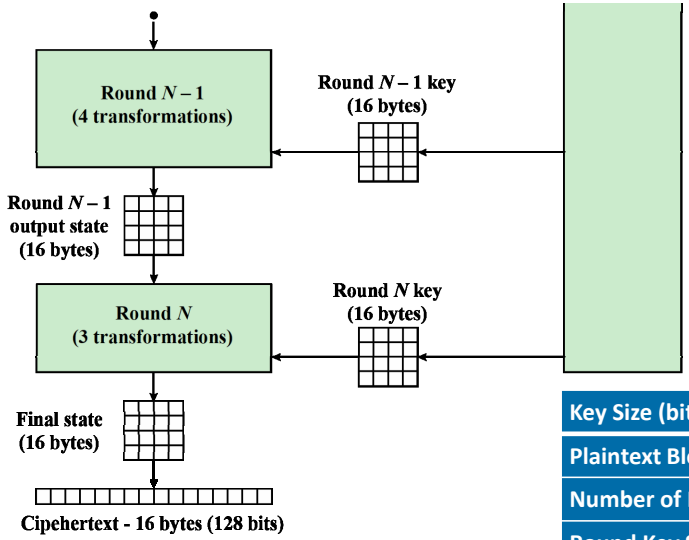
Joan Daemen en Vincent Rijmen

AES general structure (1/2)



No. of rounds	Key Length (bytes)
10	16
12	24
14	32

AES general structure (2/2)



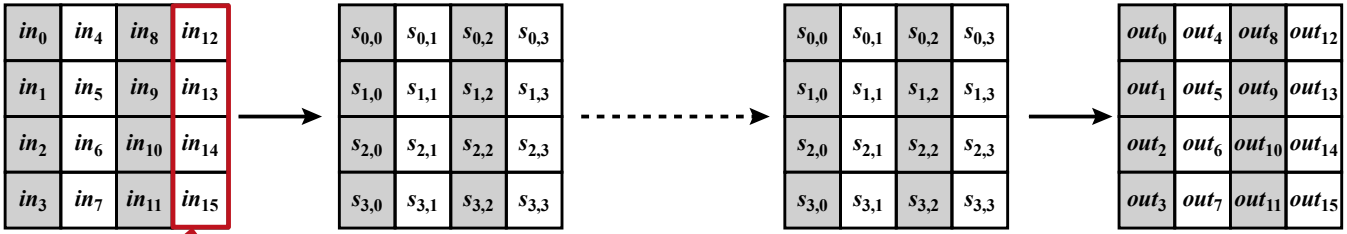
No. of rounds	Key Length (bytes)
10	16
12	24
14	32

number of rounds depends on key size

Key Size (bits)	128	192	256
Plaintext Block Size (bits)	128	128	128
Number of Rounds	10	12	14
Round Key Size (bits)	128	128	128
Expanded Key Size (bytes)	176	208	240

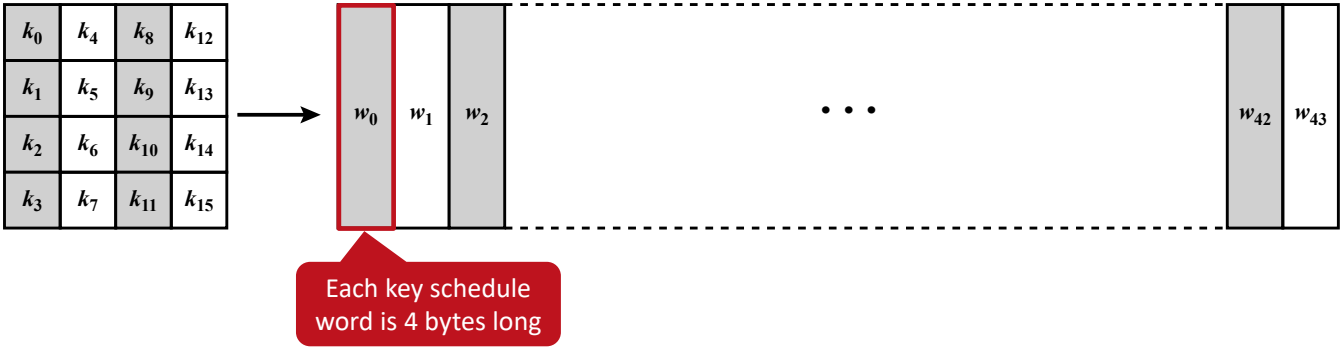
The input is represented as a 4x4 byte matrix

The input is copied into the State array, which is modified at each stage and at the final stage copied into the output matrix

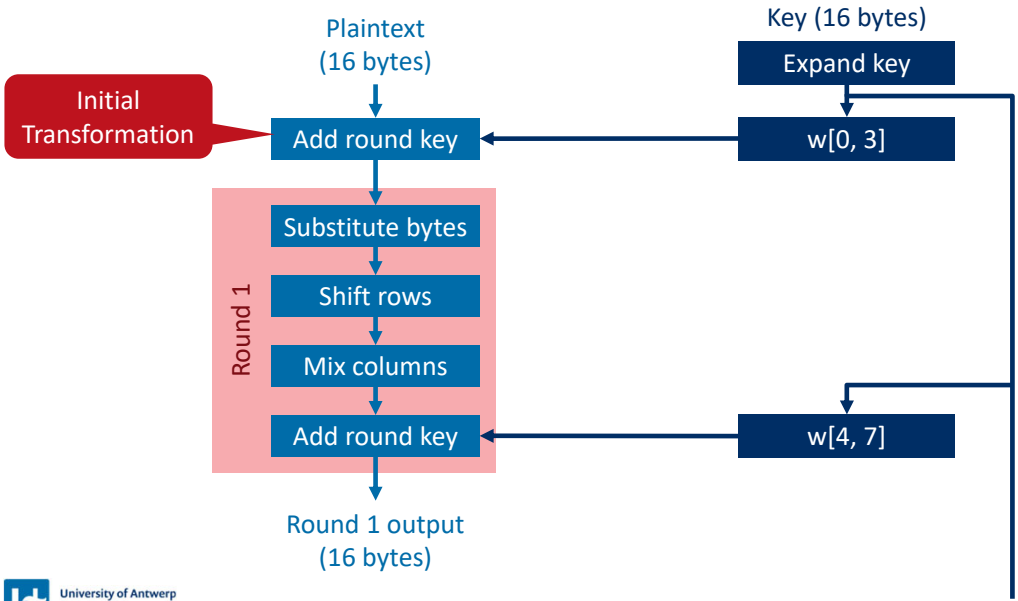


The key is also represented by a matrix

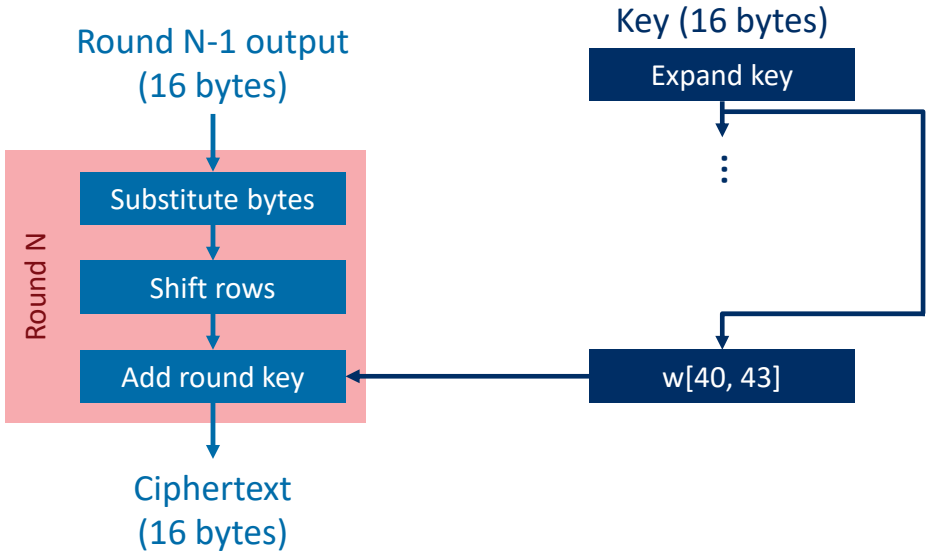
- The key is expanded into an array of key schedule words
- The key is expanded into 44, 52 or 60 words for a 128, 192 and 256-bit key respectively
- Four distinct words are combined into a 128-bit key each round



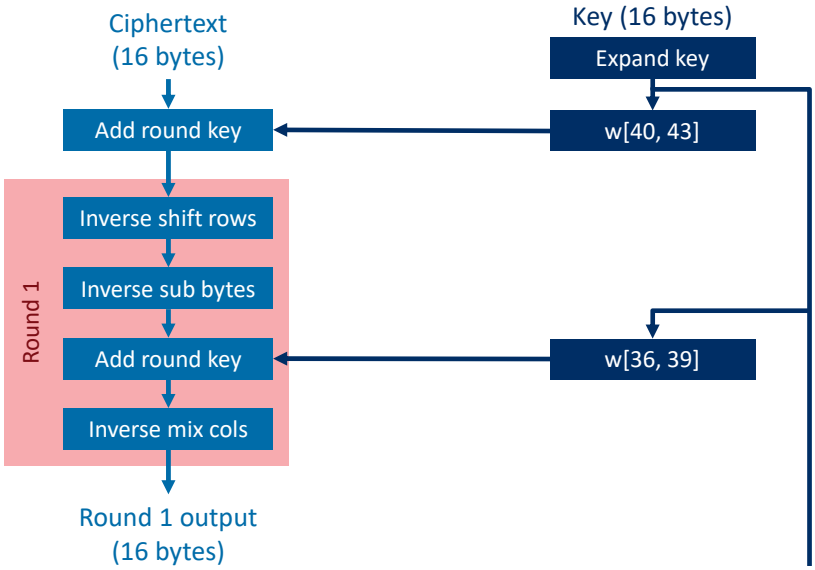
Each round consists of 4 steps (except the last)



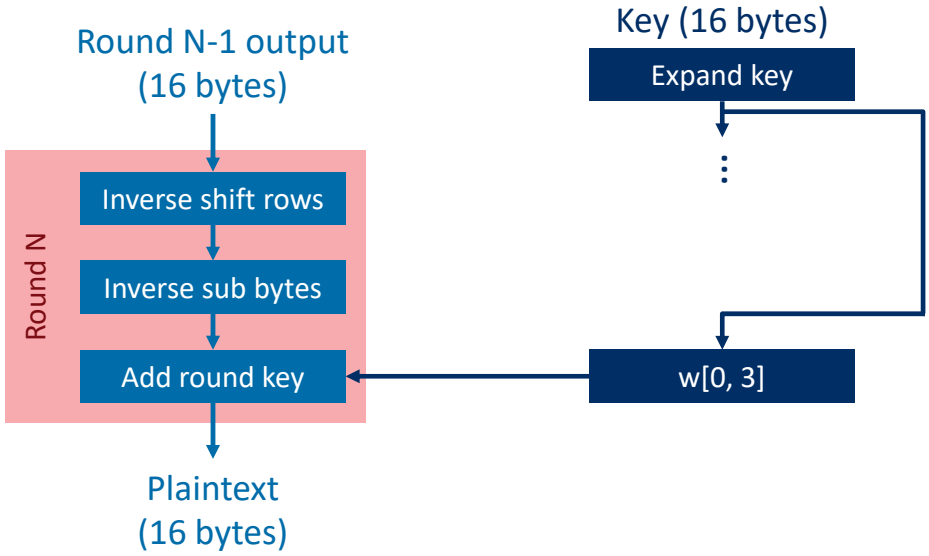
The last round has only 3 steps (no mix columns)



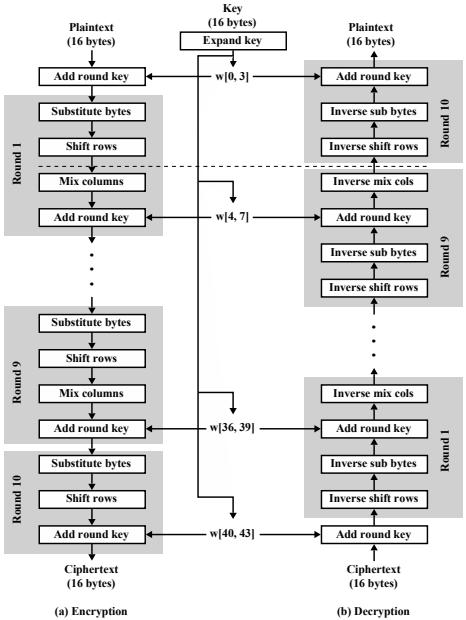
A general decryption round



The last round of decryption



Encryption and decryption are each other's inverse

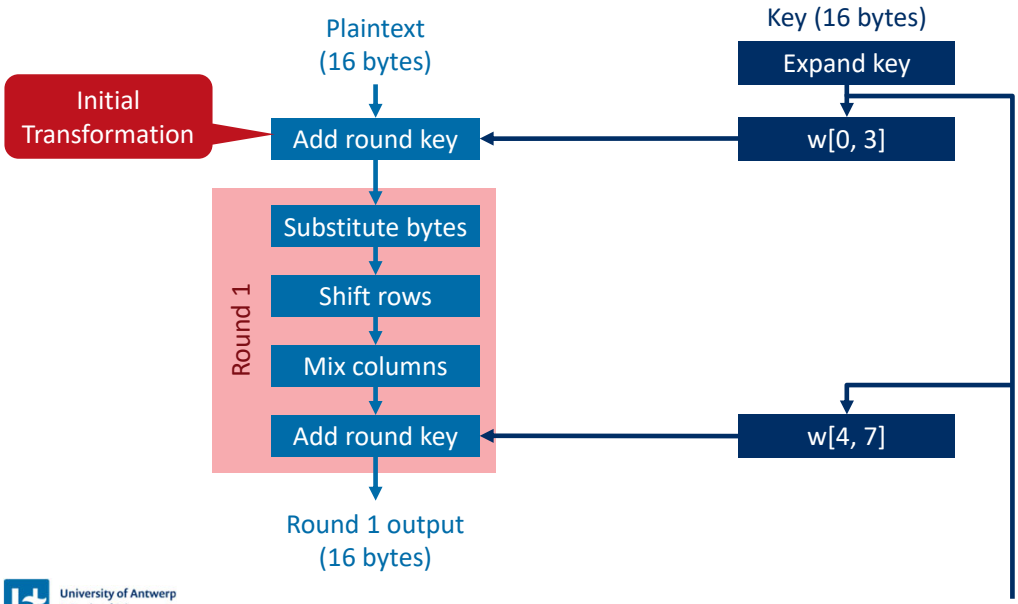


1. General block cipher structure
2. Advanced Encryption Standard (AES)
3. AES details

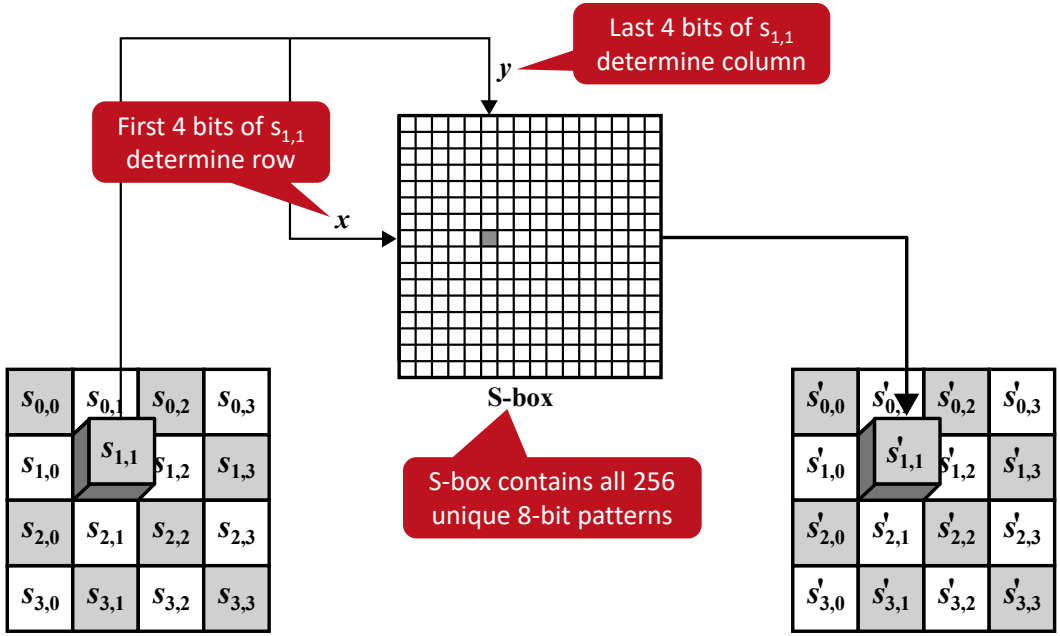


Check Homework slides on Finite Fields for mathematical background

Each round in AES consists of 4 operations



Step 1: SubBytes



S-box and inverse S-box

x \ y	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	63	7C	77	7B	F2	6B	6F	C5	30	01	67	2B	FE	D7	AB	76
1	CA	82	C9	7D	FA	59	47	F0	AD	D4	A2	AF	9C	A4	72	C0
2	B7	FD	93	26	36	3F	F7	CC	34	A5	E5	F1	71	D8	31	15
3	04	C7	23	C3	18	96	05	9A	07	12	80	E2	EB	27	B2	75
4	09	83	2C	1A	1B	6E	5A	A0	52	3B	D6	B3	29	E3	2F	84
5	53	D1	00	ED	20	FC	B1	5B	6A	CB	BE	39	4A	4C	58	CF
6	D0	EF	AA	FB	43	4D	33	85	45	F9	02	7F	50	3C	9F	A8
7	51	A3	40	8F	92	9D	38	F5	BC	B6	DA	21	10	FF	F3	D2
8	CD	0C	13	EC	5F	97	44	17	C4	A7	7E	3D	64	5D	19	73
9	60	81	4F	DC	22	2A	90	88	46	EE	B8	14	DE	5E	0B	DB
A	E0	32	3A	0A	49	06	24	5C	C2	D3	AC	62	91	95	E4	79
B	E7	C8	37	6D	8D	D5	4E	A9	6C	56	F4	EA	65	7A	AE	08
C	BA	78	25	2E	1C	A6	B4	C6	E8	DD	74	1F	4B	BD	8B	8A
D	70	3E	B5	66	48	03	F6	0E	61	35	57	B9	86	C1	1D	9E
E	E1	F8	98	11	69	D9	8E	94	9B	1E	87	E9	CE	55	28	DF
F	8C	A1	89	0D	BF	E6	42	68	41	99	2D	0F	B0	54	BB	16

S-box

S-box(29) = A5
IS-box(A5) = 29

Inverse S-box (IS-box)

x \ y	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	52	09	6A	D5	30	36	A5	38	BF	40	A3	9E	81	F3	D7	FB
1	7C	E3	39	82	9B	2F	FF	87	34	8E	43	44	C4	DE	E9	CB
2	54	7B	94	32	A6	C2	23	3D	EE	4C	95	0B	42	FA	C3	4E
3	08	2E	A1	66	28	D9	24	B2	76	5B	A2	49	6D	8B	D1	25
4	72	F8	F6	64	86	68	98	16	D4	A4	5C	CC	5D	65	B6	92
5	6C	70	48	50	FD	ED	B9	DA	5E	15	46	57	A7	8D	9D	84
6	90	D8	AB	00	8C	BC	D3	0A	F7	E4	58	05	B8	B3	45	06
7	D0	2C	1E	8F	CA	3F	0F	02	C1	AF	BD	03	01	13	8A	6B
8	3A	91	11	41	4F	67	DC	EA	97	F2	CF	CE	F0	B4	E6	73
9	96	AC	74	22	E7	AD	35	85	E2	F9	37	E8	1C	75	DF	6E
A	47	F1	1A	71	1D	29	C5	89	6F	B7	62	0E	AA	18	BE	1B
B	FC	56	3E	4B	C6	D2	79	20	9A	DB	C0	FE	78	CD	5A	F4
C	1F	DD	A8	33	88	07	C7	31	B1	12	10	59	27	80	EC	5F
D	60	51	7F	A9	19	B5	4A	0D	2D	E5	7A	9F	93	C9	9C	EF
E	A0	E0	3B	4D	AE	2A	F5	B0	C8	EB	BB	3C	83	53	99	61
F	17	2B	04	7E	BA	77	D6	26	E1	69	14	63	55	21	0C	7D

An example of substitute bytes transformation

Both x (row) and y (column) of the 8-bit input can be transformed into a two-digit hexadecimal number (4 bits), where each digit determines the row or column, which in turn are each also represented by a single digit hexadecimal number

EA	04	65	85
83	45	5D	96
5C	33	98	B0
F0	2D	AD	C5

→

87	F2	4D	97
EC	6E	4C	90
4A	C3	46	E7
8C	D8	95	A6

Rationale for S-box design

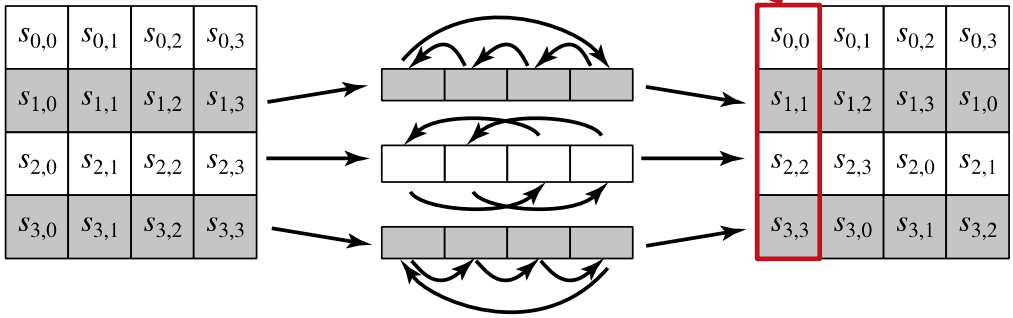
Designed to be resistant to known cryptanalytic attacks:

- **Low correlation** between input and output bits
- Output is **not a linear** mathematical function of input
- S-box has **no fixed points**: S-box(a) ≠ a
- S-box has no opposite fixed points: S-box(a) ≠ $\bar{a}$ (with $\bar{a}$ bitwise complement of a)
- S-box is **invertible**: IS-box(S-box(a)) = a
- S-box does **not self-inverse**: S-box(a) ≠ IS-box(a)

$\bar{a}$ = bitwise complement of a

Step 2: ShiftRows

Ensures the bytes of each column are spread across all columns



Example

87	F2	4D	97
EC	6E	4C	90
4A	C3	46	E7
8C	D8	95	A6

→

87	F2	4D	97
6E	4C	90	EC
46	E7	4A	C3
A6	8C	D8	95

Step 3: MixColumns

$$\begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} \begin{bmatrix} s_{0,0} & s_{0,1} & s_{0,2} & s_{0,3} \\ s_{1,0} & s_{1,1} & s_{1,2} & s_{1,3} \\ s_{2,0} & s_{2,1} & s_{2,2} & s_{2,3} \\ s_{3,0} & s_{3,1} & s_{3,2} & s_{3,3} \end{bmatrix} = \begin{bmatrix} s'_{0,0} & s'_{0,1} & s'_{0,2} & s'_{0,3} \\ s'_{1,0} & s'_{1,1} & s'_{1,2} & s'_{1,3} \\ s'_{2,0} & s'_{2,1} & s'_{2,2} & s'_{2,3} \\ s'_{3,0} & s'_{3,1} & s'_{3,2} & s'_{3,3} \end{bmatrix}$$

Small hexadecimal values for fast calculation

16-byte input state

Sum of products performed in GF(2^8)

The MixColumns transformation on a column can be expressed as:

$$s'_{0,j} = (2 \cdot s_{0,j}) \oplus (3 \cdot s_{1,j}) \oplus s_{2,j} \oplus s_{3,j}$$
$$s'_{1,j} = s_{0,j} \oplus (2 \cdot s_{1,j}) \oplus (3 \cdot s_{2,j} \oplus s_{3,j}$$
$$s'_{2,j} = s_{0,j} \oplus s_{1,j} \oplus (2 \cdot s_{2,j}) \oplus (3 \cdot s_{3,j})$$
$$s'_{3,j} = (3 \cdot s_{0,j}) \oplus s_{1,j} \oplus s_{2,j} \oplus (2 \cdot s_{3,j})$$

Exercise: Calculating the MixColumns output state

Exercise: Calculate the output state element $s'_{0,0}$ of the MixColumns transformation given the input state below.

$$\begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} \begin{bmatrix} s_{0,0} & s_{0,1} & s_{0,2} & s_{0,3} \\ s_{1,0} & s_{1,1} & s_{1,2} & s_{1,3} \\ s_{2,0} & s_{2,1} & s_{2,2} & s_{2,3} \\ s_{3,0} & s_{3,1} & s_{3,2} & s_{3,3} \end{bmatrix} = \begin{bmatrix} s'_{0,0} & s'_{0,1} & s'_{0,2} & s'_{0,3} \\ s'_{1,0} & s'_{1,1} & s'_{1,2} & s'_{1,3} \\ s'_{2,0} & s'_{2,1} & s'_{2,2} & s'_{2,3} \\ s'_{3,0} & s'_{3,1} & s'_{3,2} & s'_{3,3} \end{bmatrix}$$

Input state:

87	F2	4D	97
6E	4C	90	EC
46	E7	4A	C3
A6	8C	D8	95

$$s'_{0,j} = (2 \cdot s_{0,j}) \oplus (3 \cdot s_{1,j}) \oplus s_{2,j} \oplus s_{3,j}$$
$$s'_{1,j} = s_{0,j} \oplus (2 \cdot s_{1,j}) \oplus (3 \cdot s_{2,j} \oplus s_{3,j}$$
$$s'_{2,j} = s_{0,j} \oplus s_{1,j} \oplus (2 \cdot s_{2,j}) \oplus (3 \cdot s_{3,j})$$
$$s'_{3,j} = (3 \cdot s_{0,j}) \oplus s_{1,j} \oplus s_{2,j} \oplus (2 \cdot s_{3,j})$$

Inverse MixColumns step

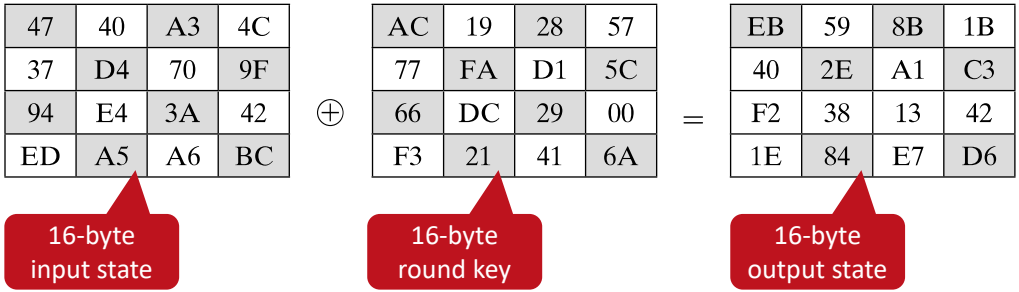
$$\begin{bmatrix} 0E & 0B & 0D & 09 \\ 09 & 0E & 0B & 0D \\ 0D & 09 & 0E & 0B \\ 0B & 0D & 09 & 0E \end{bmatrix} \begin{bmatrix} s_{0,0} & s_{0,1} & s_{0,2} & s_{0,3} \\ s_{1,0} & s_{1,1} & s_{1,2} & s_{1,3} \\ s_{2,0} & s_{2,1} & s_{2,2} & s_{2,3} \\ s_{3,0} & s_{3,1} & s_{3,2} & s_{3,3} \end{bmatrix} = \begin{bmatrix} s'_{0,0} & s'_{0,1} & s'_{0,2} & s'_{0,3} \\ s'_{1,0} & s'_{1,1} & s'_{1,2} & s'_{1,3} \\ s''_{2,0} & s'_{2,1} & s'_{2,2} & s'_{2,3} \\ s'_{3,0} & s'_{3,1} & s'_{3,2} & s'_{3,3} \end{bmatrix}$$

Larger hexadecimal values, means decryption is computationally harder than encryption

Step 4: AddRoundKey

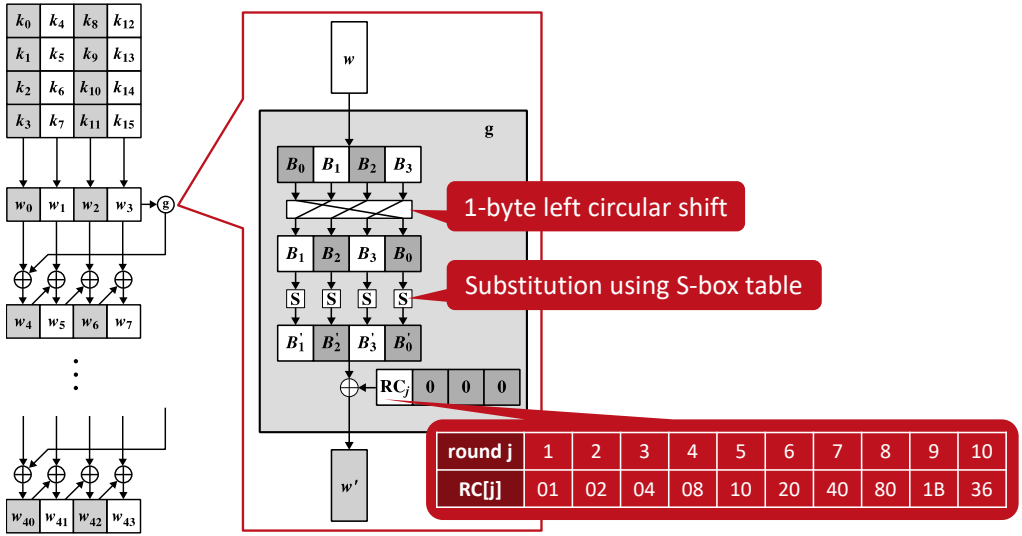
Bitwise XOR of 128-bit input state with 128-bit round key

Example



AES key expansion

Transform key into linear array of 44, 52 or 60 words



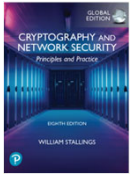
Rationale for key generation algorithm

- Knowledge of a part of the cipher key or round keys does not enable calculation of many other round-key bits
- The transformation is invertible
- Computationally efficient on many processors
- The round constants RC[j] eliminate symmetry between round keys
- Each cipher key bit affects many round key bits (diffusion)
- Nonlinear to prohibit determination of round key differences based on cipher key differences
- Simple description

Summary of symmetric ciphers

Cipher	Type	Characteristics
Caesar	Substitution	Extremely vulnerable to **brute force attacks**
Monoalphabetic	Substitution	Vulnerable to statistical **frequency attacks**
Vigenère	Substitution	Limited vulnerability to **frequency attacks**
One-time pad	Substitution	**Unconditionally secure**, but **practically infeasible**
RC4	Substitution	Not secure due to **bias in PRG**
Salsa20	Substitution	**Computationally secure** with key size of 256 bits
DES	Hybrid	Vulnerable to **brute force** due to 56-bit key size
Triple DES	Hybrid	**Computationally secure** with key size of 168 bits
AES	Hybrid	**Computationally secure**

Further reading on block ciphers



- Chapter 4 (Section 4.1, 4.2)
- Chapter 6 (Section 6.1—6.4)
- Chapter 7 (Section 7.1)



- Block ciphers (videos 1, 2 and 3)

Block operation modes

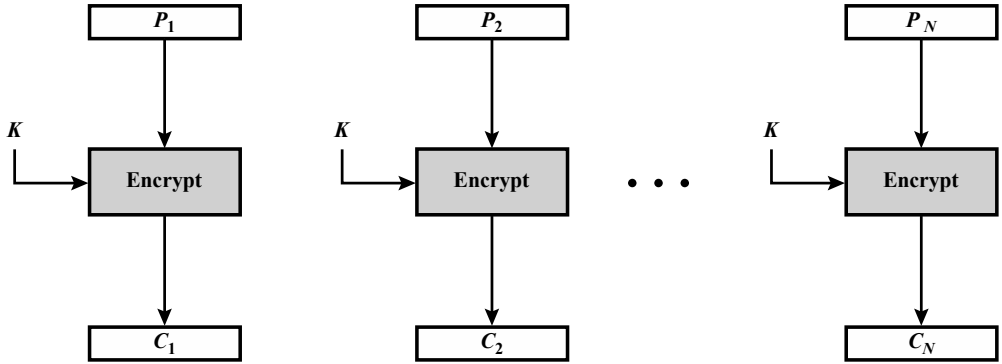
NIST modes of operation for block ciphers

- When the plaintext size is larger than the block size B, the text is split into fixed-length blocks, all encrypted with the same key
  - Encryption of multiple blocks with the same key reduces cryptographic security
- NIST defined block cipher operation modes to solve this issue
  - 1981: NIST defined ECB, CBC, OFB and CFB for DES in FIPS 81 (“DES Modes of Operation”)
  - 2001: Support for CTR mode and AES in SP800-38A (“Recom. for Block Cipher Modes of Operation”)
  - 2010: Support for XTS-AES in SP800-38E (“The XTS-AES Mode for Confidentiality on Storage Devices”)
- Modes can in principle be applied to any type of block cipher

Mode	Typical Application
Electronic Codebook (ECB)	• Secure transmission of one value (e.g., encryption key)
Cipher Block Chaining (CBC)	• General-purpose block-oriented transmission • Authentication
Cipher Feedback (CFB)	• General-purpose stream-oriented transmission • Authentication
Output Feedback (OFB)	• Stream-oriented transmission over noisy channel (e.g., satellite communication)
Counter (CTR)	• General-purpose block-oriented transmission • Useful for high-speed requirements
XEX-based Tweaked-codebook mode with ciphertext Stealing (XTS-AES)	• Developed specifically for sector-based storage devices • Protects against adversaries that have access to the stored data

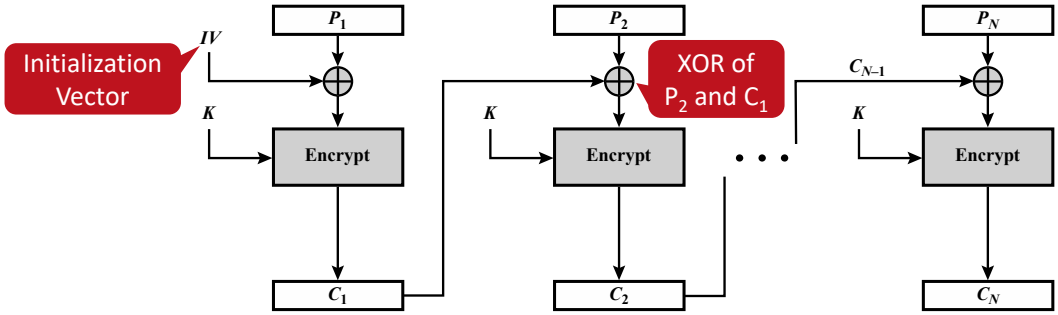
Electronic codebook (ECB)

Simplest form, where each block of plaintext is encoded independently using the same key.



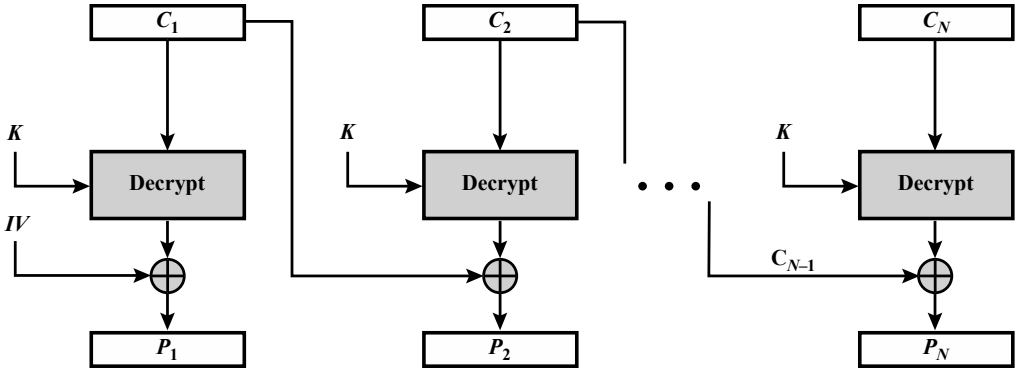
Cipher Block Chaining (CBC)

Makes sure encrypting the same plaintext block with the same key does not result in the same ciphertext block (in contrast to ECB)



IV must be known to sender and receiver, but kept hidden from third parties (e.g., using ECB encryption)

Cipher Block Chaining (CBC) decryption



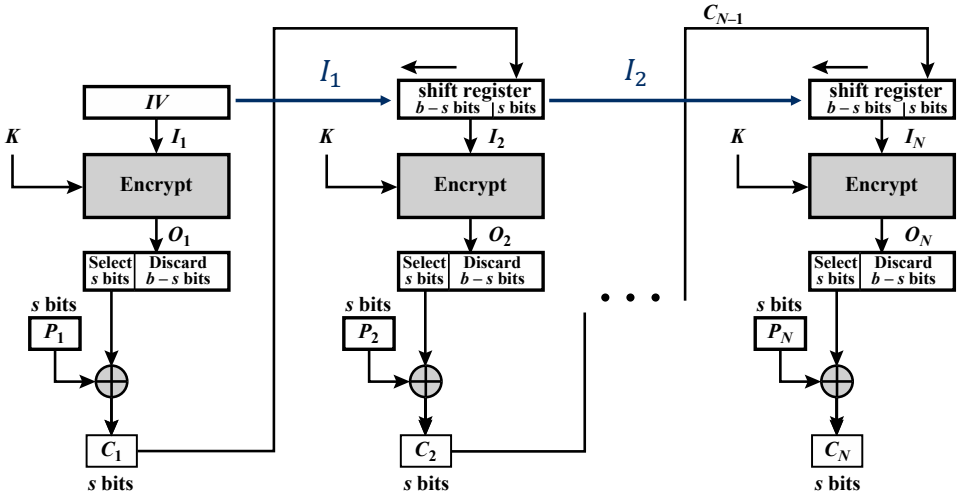
Cipher Block Chaining (CBC) proof of equivalence

Proof of equivalence (i.e., decryption is the inverse of encryption)

$$
\begin{aligned}
& P_j = D(K, C_j) \oplus C_{j-1} \\
\Leftrightarrow\ & P_j \oplus C_{j-1} = D(K, C_j) \oplus C_{j-1} \oplus C_{j-1} \\
\Leftrightarrow\ & E(K, P_j \oplus C_{j-1}) = E(K, D(K, C_j) \oplus C_{j-1} \oplus C_{j-1}) \\
\Leftrightarrow\ & E(K, P_j \oplus C_{j-1}) = E\left(K, D(K, C_j)\right) \\
\Leftrightarrow\ & E(K, P_j \oplus C_{j-1}) = C_j
\end{aligned}
$$

Cipher Feedback (CFB)

- A stream cipher mode that operates on small blocks of s bits (e.g., 8)
- Starting from an IV of b bits (the original block size)

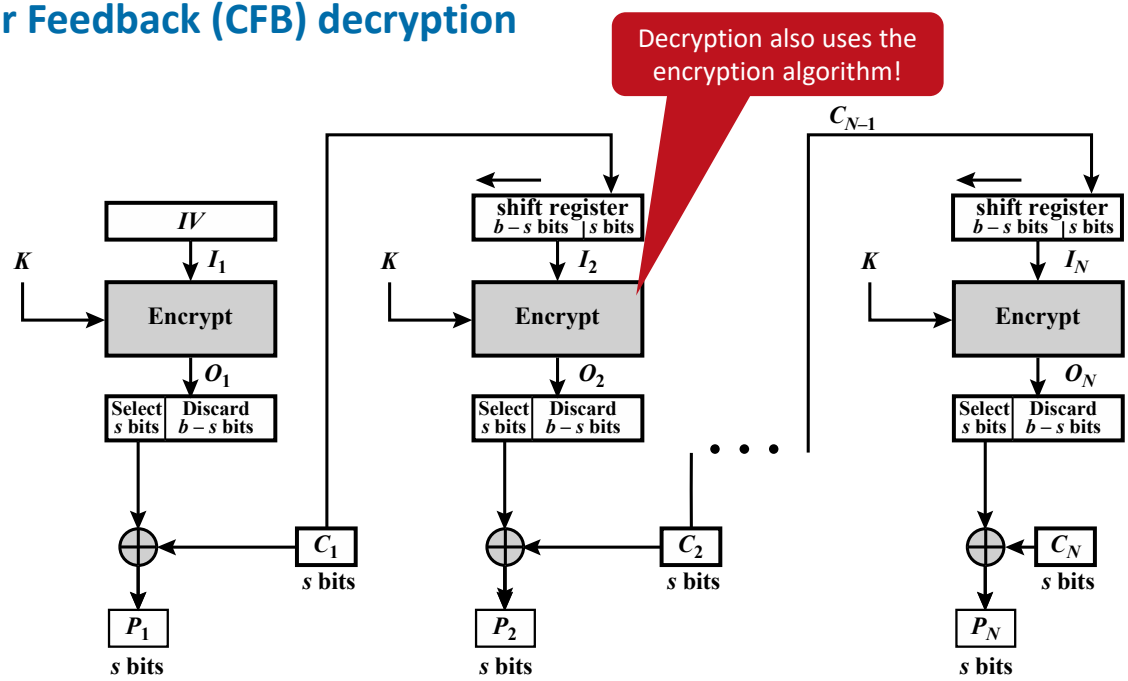


$$I_j = LSB_{b-s}(I_{j-1}) \,||\, C_{j-1}$$

$$C_j = MSB_s\left(E(K, I_j)\right) \oplus P_j$$

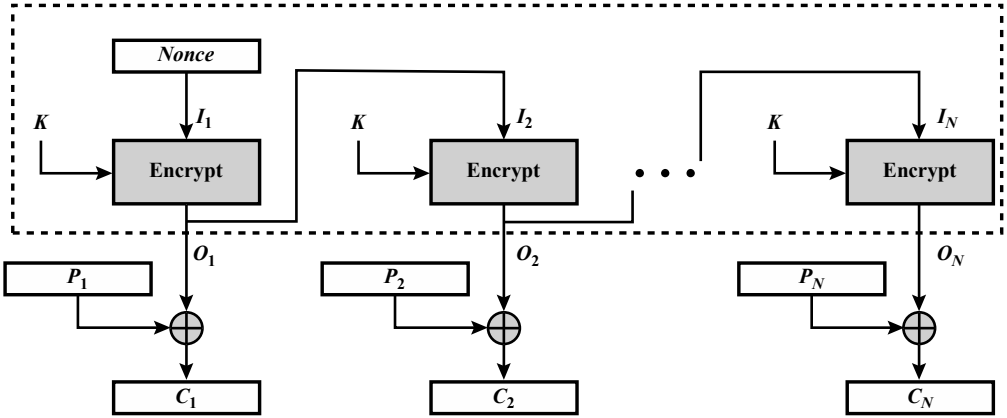
LSB_x = x least significant bits;
MSB_x = x most significant bits

Cipher Feedback (CFB) decryption



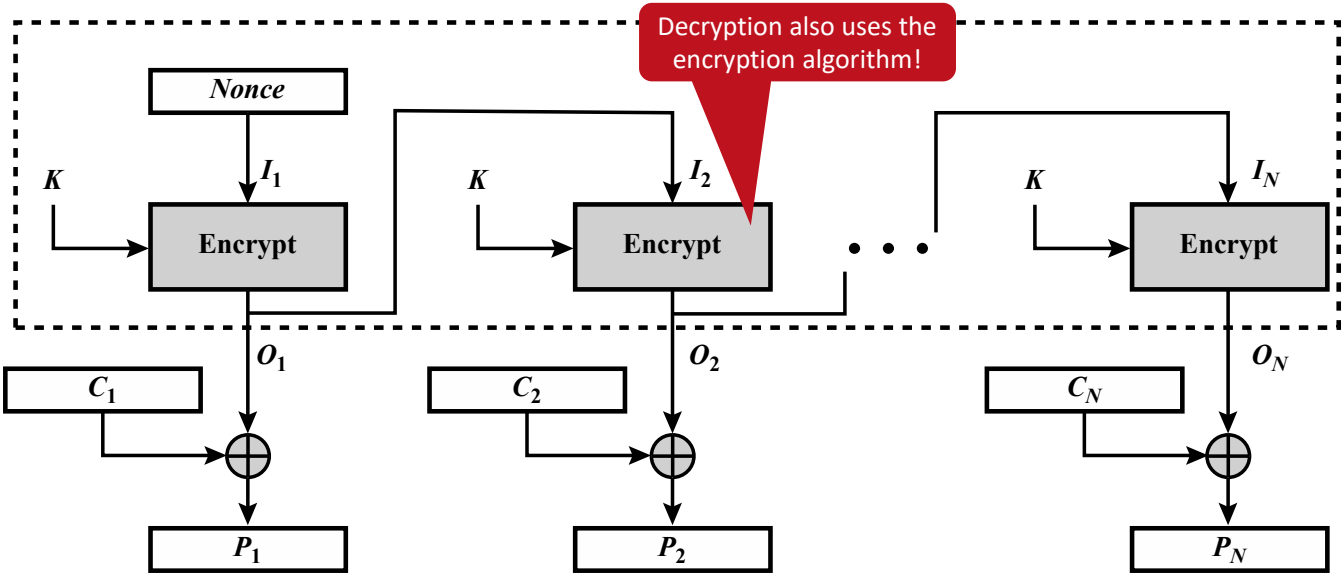
Output feedback (OFB)

Similar to CFB, but instead operates on full blocks of size b



Nonce is an IV that must be unique for each execution, since the encryption of a block does not depend on the plaintext P, but only on the key K and the nonce itself.

Output feedback (OFB) decryption



When poll is active respond at **PollEv.com/jfamaey** Send **jfamaey** to **+32 460 20 00 56**

Which block cipher feedback mode is most vulnerable to transmission bit errors in the ciphertext?

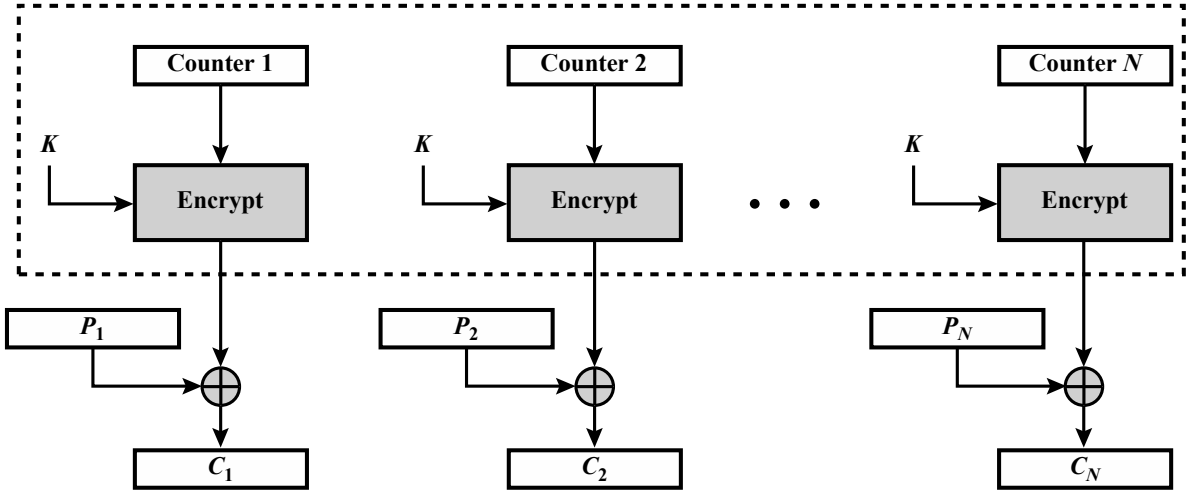
0

- Cipher Feedback Mode (CFB)
- Output Feedback Mode (OFB)
- Both are equally vulnerable
- [illegible]

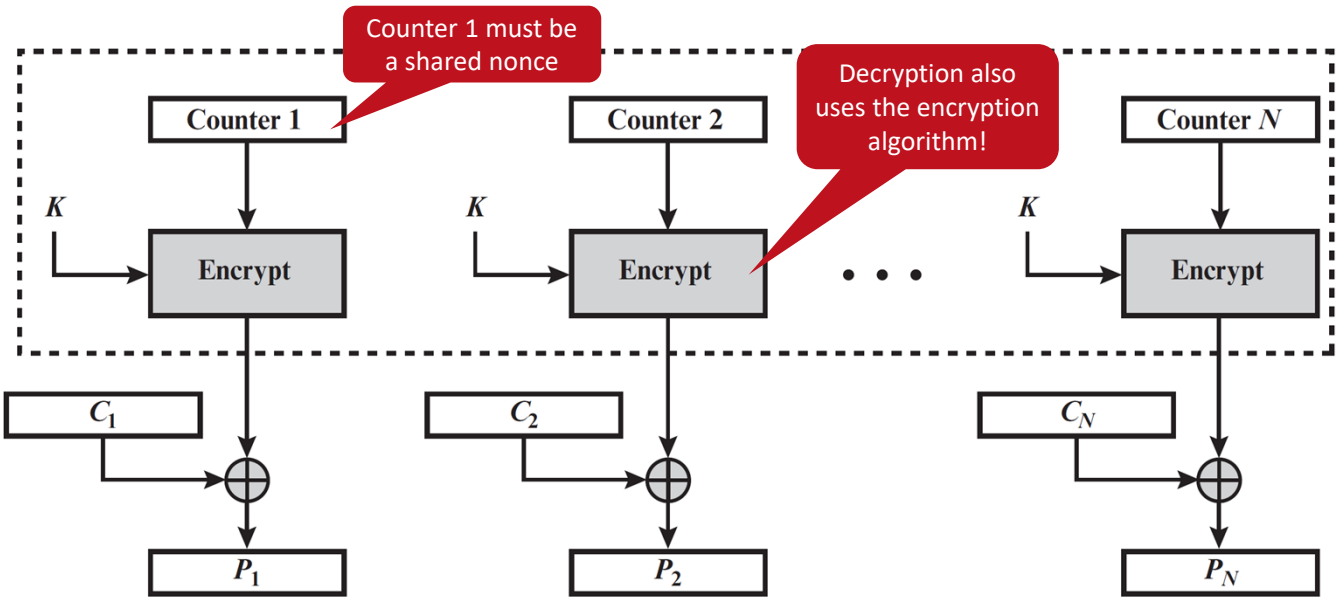
Instructions | Responses | Correctness | More | Clear responses

Counter (CTR)

Uses a counter equal to the plaintext block size, which must be different for each encrypted plaintext block.



Counter (CTR) decryption



Advantages of Counter mode

Hard- and software efficiency
As there is no chaining, encryption of blocks can be done in parallel

Pre-processing
Encryption only depends on the counter, and can therefore be done in advance

Random access
As there is no chaining, the ith block can be accessed in random access fashion

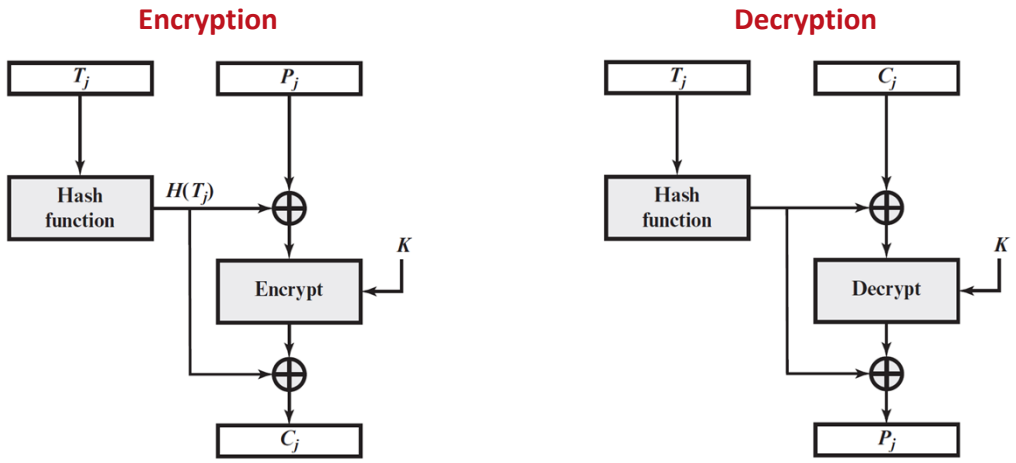
Proven security
If a unique counter is used for each block, CTR is at least as secure as other modes

Simplicity
CTR (like CFB and OFB) only requires the implementation of the encryption algorithm

XEX-based Tweaked-codebook mode with ciphertext Stealing

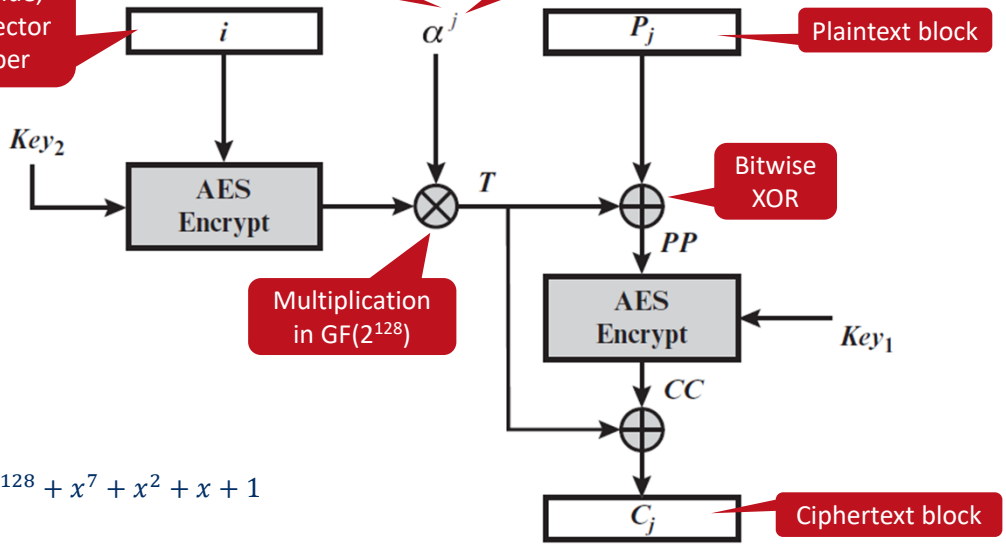
- For encryption of data stored on sector-based devices
- First standardized by IEEE in 2007 (Std. 1619-2007) and later by NIST in 2010
- Based on the concept of **tweakable block ciphers**, that uses as input a (non-secret) tweak T in addition to the secret symmetric key K
- Requirements for “data at rest” differ from those of transmitted data
  1. The ciphertext is freely available for an attacker
  2. The encrypted data must be the same size as the plaintext data
  3. Data are accessed in fixed sized blocks, independently from each other
  4. Encryption is performed in 16-byte blocks, independent from other blocks
  5. There are no other metadata used, except the location of the data blocks
  6. The same plaintext is encrypted to different ciphertexts at different locations, but the same ciphertext in the same location
  7. A standard conformant device can be constructed for decryption of data encrypted by another standard conformant device

General structure of tweakable block ciphers



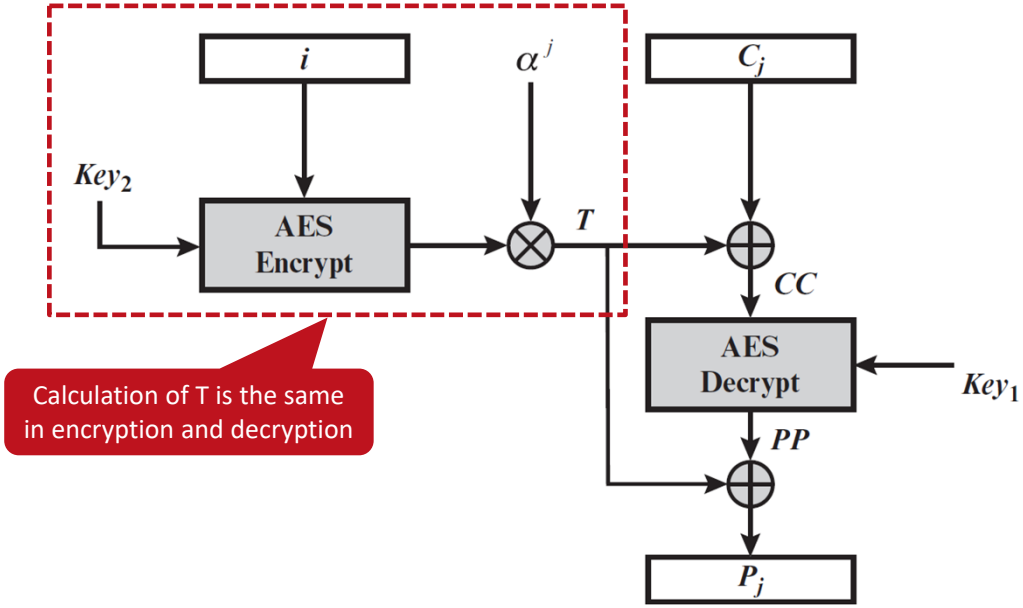
A block cipher mode of operation can be created by using a different tweak value on each block.

XTS-AES encryption of a single block



GF(2^{128}): $m(x) = x^{128} + x^7 + x^2 + x + 1$

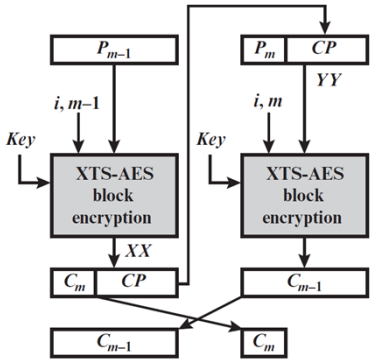
XTS-AES decryption of a single block



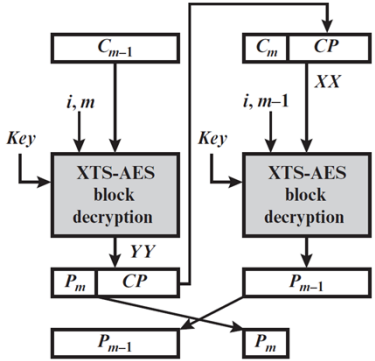
XTS-AES operations on a full sector

- Each sector consists of a set of 128-bit blocks labelled P_0, P_1, ..., P_m
- All blocks, except the last 2, are encrypted independently (cf., previous slides)
- If the last block contains less than 128 bits, **ciphertext-stealing** is applied

Encryption of P_{m-1} and P_m



Decryption of C_{m-1} and C_m

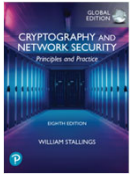


Summary of block cipher operational modes

- Block cipher operation modes:
  - Ensure that the same plaintext block does not result in same ciphertext block
  - CFB mode allows block ciphers to be used as stream ciphers
  - CTR mode is computationally efficient and easy to implement
  - XTS-AES mode is optimized for the peculiarities of encrypting data-at-rest

Mode	Typical Application
Electronic Codebook (ECB)	• Secure transmission of one values (e.g., encryption key)
Cipher Block Chaining (CBC)	• General-purpose block-oriented transmission • Authentication
Cipher Feedback (CFB)	• General-purpose stream-oriented transmission • Authentication
Output Feedback (OFB)	• Stream-oriented transmission over noisy channel (e.g., satellite communication)
Counter (CTR)	• General-purpose block-oriented transmission • Useful for high-speed requirements
XEX-based Tweaked-codebook mode with ciphertext Stealing (XTS-AES)	• Developed specifically for sector-based storage devices • Protects against adversaries that have access to the stored data

Further reading on block cipher operational modes



- Chapter 7 (Section 7.2—7.7)



- How to Use Block Ciphers 1
  - Modes of operation: one-time key
- How to Use Block Ciphers 2
  - CBC
  - CTR

End of Part 2